

- -----
- -----CI/CD with GitLab-----
 - ----Introduction
 -  1. CI/CD Overview
 -  2. Continuous Integration (CI)
 - Key Steps:
 -  3. Continuous Delivery (CD)
 -  4. Continuous Deployment (CD)
 -  5. CI/CD Pipeline – Example Flow
 -  6. Popular CI/CD Tools
 -  7. Benefits of CI/CD
 -  8. Challenges
 -  Summary Table
 - ----GitLab in comparison to other CI/CD platforms
 -  1. GitLab Overview
 -  2. Comparison with Other Platforms
 -  In the context of GitLab
 -  Analogy
 -  Why choose self-hosted GitLab?
 -  Drawbacks
 -  3. GitLab's CI/CD Strengths
 -  4. GitLab Limitations
 -  5. When to Choose GitLab
 - ----GitLab vs GitHub
 -  Similarities (Both Can Do These)
 -  Differences – Why Some Prefer GitHub
 -  Why Some Companies Use GitLab Instead
 -  When to Choose Which?
 - ----GitLab vs GitHub Actions for CI/CD
 -  1. Core Concept
 -  2. Configuration
 -  3. Hosting
 -  4. Features
 -  5. Pricing
 -  6. Enterprise Use Cases
 -  Summary
- ----GitLab Pipeline
 -  GitLab CI/CD Flow
 -  Key Components of GitLab Pipelines
 -  Example .gitlab-ci.yml
 -  How Pipelines Run
 -  Types of GitLab Pipelines

-  Steps to Create a Pipeline in GitLab GUI
 - Example Using GUI (Generated YAML)
 -  So, summary :
 -  Benefits of GitLab Pipelines
- ----GitLab Runners (For more Info see -- Keyword - tags)
 -  Types of GitLab Runners
 -  Runner Executors
 -  How Runners Work (Flow)
 -  Installing a GitLab Runner
 -  Example Runner Configuration
 -  Using Tags to Target Runners
 -  Key Benefits of Custom Runners
 -  In short :
- ----Multiple Jobs
 - 1. Basic Structure of Multiple Jobs
 -  How this runs:
 - 2. Multiple Jobs Running in Parallel
 -  How this runs:
 - 3. Mix of Sequential + Parallel
 -  Summary :
- ----Keyword - before_script and after_script
 -  before_script
 -  after_script
 -  Combined Example with Multiple Jobs
- ----GitLab Artifacts
 -  Defining Artifacts in .gitlab-ci.yml
 -  Example:
 -  Artifact Expiration
 -  Artifact Passing Between Jobs
 - Example: Sequential passing
 -  Special Reports (Test, Coverage, etc.)
 -  Downloading Artifacts
 -  Summary :
- ----Environmental Variables in GitLab
 -  1. Adding Environment Variables in GitLab UI (Recommended for Secrets)
 -  2. Adding Variables in .gitlab-ci.yml
 -  3. Passing Variables at Runtime (Trigger or Schedule)
 -  4. Using Variables in Scripts
 -  Summary :
- ----Email Notifications in GitLab
 -  How Email Notifications Work in GitLab
 -  Configuring Email Notifications
 -  Pipeline/Job Email Notifications

-  Admin Setup (SMTP)
-  Summary
- ----Scheduled Pipeline
 -  How Scheduling Works in GitLab
 -  Cron Syntax Refresher
 -  Example .gitlab-ci.yml with Scheduled Job
 -  Setting Up a Schedule in GitLab (GUI)
 -  Custom Environment Variables for Scheduled Jobs
 -  Sequential vs Parallel Schedules
 -  In summary :
- ----Running Pipelines Manually
- ----Manual Jobs and Manual Deployment
 -  What is Manual Deployment in GitLab?
 -  How to Define Manual Jobs
 -  Optional vs Required Manual Jobs
 -  Manual Jobs in Multi-Stage Pipelines
 -  Combining with Scheduled Pipelines
 -  Permissions
 -  Triggering Manual Jobs
- ----Predefined Variables in GitLab
 -  What are Predefined Variables in GitLab?
 -  Categories of Predefined Variables
 -  Example Usage in .gitlab-ci.yml
 -  Why are Predefined Variables Useful?
- ----Importing Github project to GittLab
 -  Why Import GitHub → GitLab?
 -  Ways to Import a GitHub Project into GitLab
 - 1. Using GitLab's GitHub Importer (Recommended)
 - 2. Manual Import via Git Clone & Push
 - 3. Using GitHub → GitLab Migration Tools
 -  Things to Consider
- ----Running Job using Custom Image
 - 1. Where Docker comes in
 - 2. If you're not using Docker
 - 3. Why use Docker images at all?
 -  Using Custom Docker Image
 -  Benefits of custom images :
- ----Hosting Static Websites directly from your GitLab repository using pages
 -  What is GitLab Pages?
 -  How it works
 -  Basic Example: Deploying a Static Website
 -  Where will the site be hosted?
 -  Key Rules for GitLab Pages Job

- ----All Stages, Jobs and reserved Keys in .gitlab-ci.yml
 -  1. Built-in Jobs in .gitlab-ci.yml
 -  2. Built-in Stages
 -  3. Built-in Keys (keywords)
 - Pipeline structure keys
 - Job-level keys
 - Special keys
 -  4. Example Full .gitlab-ci.yml Using Many Keys
 -  Summary:
- ----Keyword - rules
 -  What is rules: in GitLab?
 -  Structure
 -  Examples
 -  Comparison: rules: vs only/except
- ----Pipeline Structure key - workflow
 -  What is workflow in GitLab?
 -  Syntax
 -  Examples
 -  Difference: workflow: rules vs job rules
 -  Summary:
- ----Keyword - include
 -  Why use include?
 -  Types of include
 -  Example – Combining multiple includes
 -  How it works
 -  Use Case Example:
- ----Keyword - tags
 -  What are Tags in GitLab CI/CD?
 -  Why Tags Are Needed?
 -  Defining Tags for a Job
 -  Registering a Runner with Tags
 -  Example Flow
 -  Key Points
 -  Why can't we just use one runner for everything ?
 - 1. Performance and Scalability
 - 2. Isolation
 - 3. Security
 - 4. Specialization
 - 5. Maintenance
 -  When does one runner make sense?
 -  When do you need multiple runners ?
 -  Best practice :
 -  More Details on Runners

- 1. Types of GitLab Runners
- 2. So can I just use 1 runner with Linux or Windows?
- ---Keyword - services
 -  What are Services in GitLab CI/CD?
 -  When do you need services?
 -  Example: Using PostgreSQL as a Service
 -  Multiple Services Example
 -  Key Points about Services
 -  So in simple words:
- ---Keyword - cache
 -  What is cache in GitLab?
 -  Difference: cache vs artifacts
 -  Syntax of cache
 - Explanation:
 -  Options for cache
 -  Example: Caching Dependencies
 -  Summary
- ---Keyword - dependencies
- ---Keyword - needs
- ---Keyword -- retry and timeout
 -  retry keyword
 - Advanced Usage (with conditions):
 -  timeout keyword
 -  Key Difference
 -  Example combining both:
- ---Keyword - environment
 -  Syntax:
 -  Example with review apps:
- ---Keyword - extends
 -  Syntax:
 - Scalars and Arrays
 -  Example:
- ---Registering a GitLab Runner on an EC2 instance (Or any server)
 -  1. Prerequisites
 -  2. Install GitLab Runner on EC2
 -  3. Register the Runner
 -  4. Configure the Runner
 -  5. Start and Enable Runner
 -  6. Verify in GitLab
- -----
- -----MongoDb Atlas-----
 - ---To Learn - Topics
 - ---Introduction

- 1. What is MongoDB Atlas?
- 2. Core Features
- 3. Cluster Architecture
- 4. Working with Atlas
- 5. Scaling in Atlas
- 6. Advanced Features
- 7. Use Cases
- 8. Benefits over Self-Hosting MongoDB
- 9. Pricing
- 10. Where to Learn Atlas Hands-On
- 🚀 IMP--You just focus on building apps (like FitLab MERN project) instead of worrying about managing databases.

• -----

• -----Hosting using EC2, NGINX, S3, Cloudfront, Route 53, GitLab-----

- 🕒 Architecture (what goes where)
- 📁 Prerequisites (one-time)
- 💻 EC2 (Ubuntu) — backend host
- 🌐 NGINX — reverse proxy + TLS + websockets
- 📦 MongoDB Atlas (prod)
- 📁 S3 — host the React build (private with OAC)
- 🚀 CloudFront — CDN for frontend (SPA-friendly)
- 🌐 Route 53 — DNS
- 🔄 GitLab CI/CD — pipelines for FE + BE
- 🖱️ Socket.IO & NGINX (important)
- 📄 WebRTC nuances (for interviews)
- 🛡️ Security checklist (prod)
- 📊 Observability
- 🔄 Rollouts & rollbacks
- ⚠️ Common pitfalls (quick fixes)
- ----Must Register Domain and Create Hosted Zone for Backend and Start with it
 - 🚩 1) Register the domain + create a hosted zone (now)
 - 💻 2) Backend first: EC2 + Nginx + DNS + SSL
 - 📁 3) Frontend: S3 + CloudFront + DNS + SSL
 - 🔄 4) Optional: Put CloudFront in front of API
 - 🕒 Summary of DNS targets (what goes where)
 - 🛡️ Certificates: two places, two tools
 - ⚠️ Common pitfalls (avoid these)
 - 👤🔍 A clean, minimal sequence you can follow
- ----Backend URL format convention
 - 1. Usual URL format for backend
 - 2. Mapping your existing local routes
 - 3. Frontend calling backend
 - 4. SSL & CORS

- ----S3 Hosting in Detail
 - 1. Build folder for S3
 - 2. .env file in Frontend
 - 3. S3 Hosting Checklist

-----CI/CD with GitLab-----

----Introduction

CI/CD – Continuous Integration & Continuous Delivery/Deployment

CI/CD is a modern software development practice that automates **building, testing, and delivering** applications so teams can release updates faster, safer, and more consistently.

1. CI/CD Overview

CI/CD has **two main components** :

- **CI (Continuous Integration)** → Developers merge code changes into a shared repository frequently, and automated builds/tests run to detect issues early.
- **CD (Continuous Delivery / Continuous Deployment)** → Automates delivering tested code to production or staging environments.

 In short:

CI makes sure your code works together.

CD makes sure it reaches users quickly and reliably.

2. Continuous Integration (CI)

Goal: Detect bugs early and ensure a working codebase at all times.

Key Steps:

1. Code Commit

- Developers push code to a central repository (GitHub, GitLab, Bitbucket).

2. Automated Build

- A CI tool (Jenkins, GitHub Actions, GitLab CI/CD, AWS CodeBuild) compiles the code and packages it.

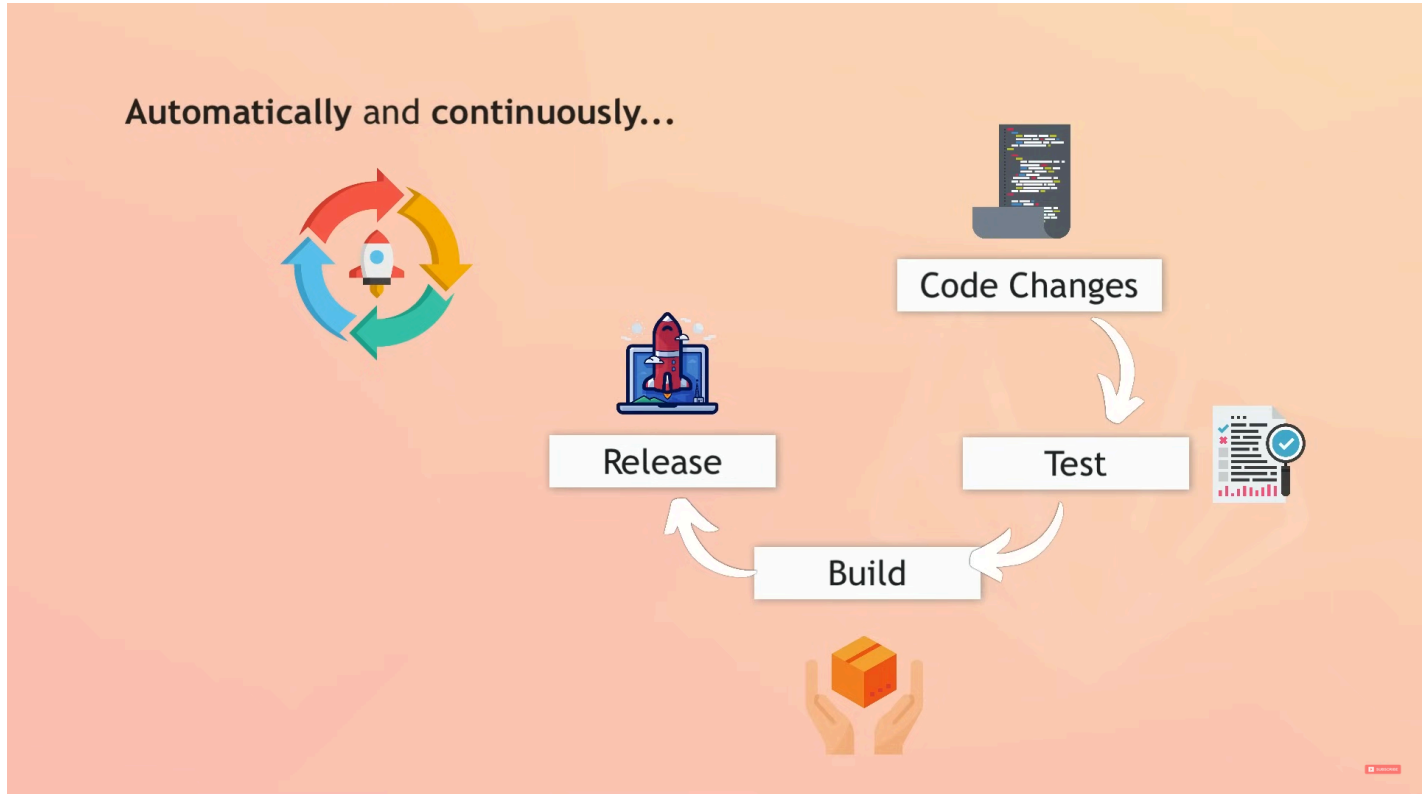
3. Automated Testing

- Unit tests, integration tests, linting, and security checks run automatically.

4. Feedback

- Developers get immediate feedback on failures before merging.

🔗 **Analogy:** Imagine multiple chefs cooking together in one kitchen — CI ensures their ingredients (code) mix well before serving the dish.



🚚 3. Continuous Delivery (CD)

Goal: Keep the code ready for production deployment **at any time** .

- After CI finishes, CD takes the tested build and deploys it to a **staging or pre-production** environment.
- Teams can manually approve and push it to **production** when ready.

🔗 **Analogy:** Like having a pizza ready in the oven; you just decide when to serve it.

⚡ 4. Continuous Deployment (CD)

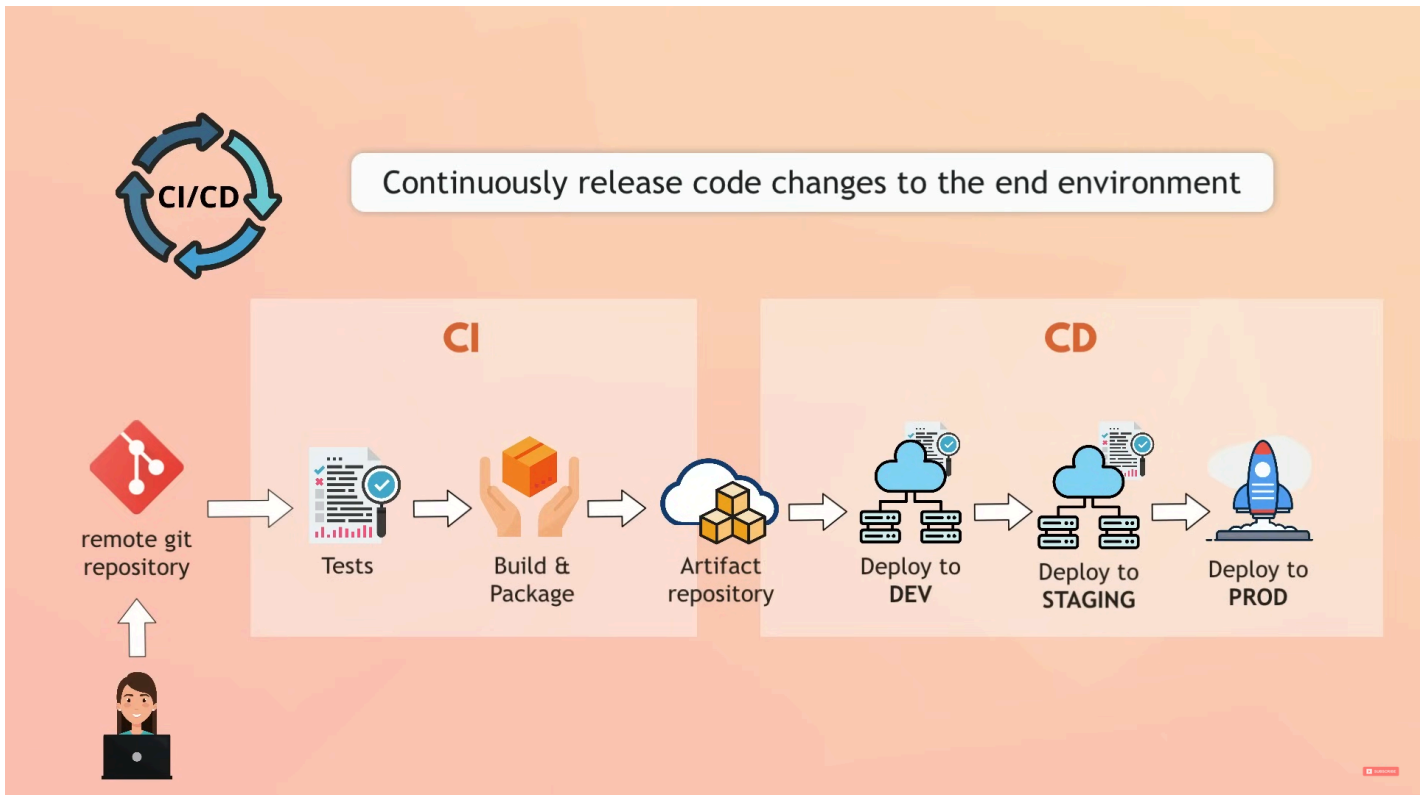
Goal: Fully automate deployment to production without manual approval.

- Every successful build from CI automatically goes live in production.
- Great for small, fast-moving teams but requires strong automated testing.

🔗 **Analogy:** As soon as the pizza is baked, it's instantly served to the customer.

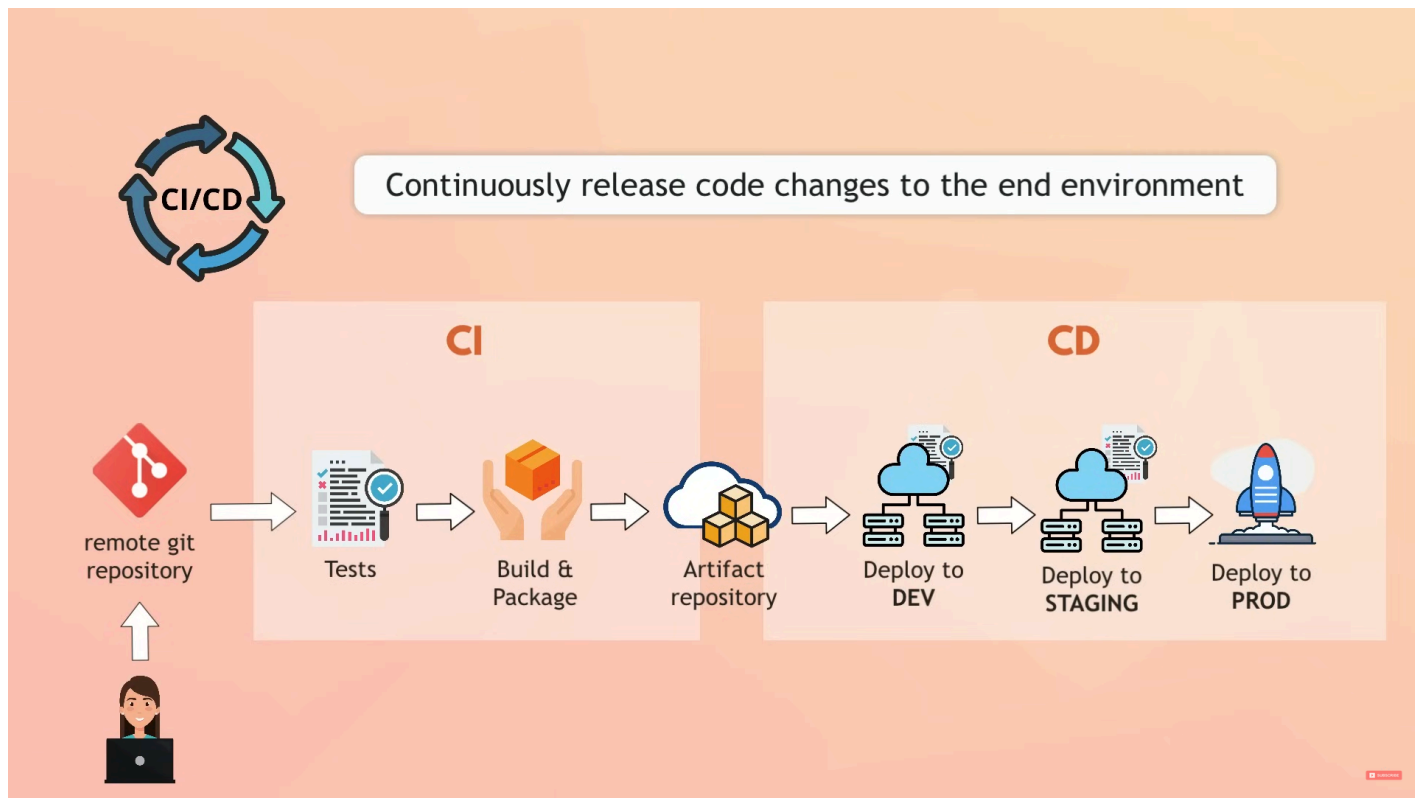
⚡ 5. CI/CD Pipeline – Example Flow

1. **Code Commit** → Developer pushes changes.
2. **Build Stage** → Code compiled, packaged.
3. **Test Stage** → Automated unit, integration, and UI tests.
4. **Deploy Stage** → To staging (Continuous Delivery) or directly to production (Continuous Deployment).
5. **Monitor Stage** → Logging, metrics, alerts for production health.



⚙️ 6. Popular CI/CD Tools

- **Jenkins** 🌀 (self-hosted, flexible, plugin-based)
- **GitHub Actions** 🍄 (integrated with GitHub repos)
- **GitLab CI/CD** 🦊 (built-in with GitLab)
- **CircleCI** 🔄 (cloud-based)
- **AWS CodePipeline** ☁️ (integrated with AWS services)
- **Azure DevOps Pipelines** 💠 (Microsoft ecosystem)



7. Benefits of CI/CD

- 🚀 Faster release cycles.
- 🐛 Early bug detection.
- 🛡️ Better security via automated checks.
- 💡 Better collaboration between dev & ops.
- 📉 Reduced deployment risk.

8. Challenges

- Requires a culture shift in the team.
- Strong automated tests are mandatory.
- Infrastructure setup and maintenance.
- Avoiding “pipeline failures” blocking releases.

✓ Summary Table

Concept	What it Does	Deployment Frequency
CI	Integrates and tests code	Multiple per day
Continuous Delivery	Code ready for release	On demand
Continuous Deployment	Code automatically released	Multiple per day

----GitLab in comparison to other CI/CD

platforms

GitLab is a **full DevOps lifecycle platform** — not just a CI/CD tool. Unlike some platforms that focus only on integration or delivery, GitLab tries to combine **source control, CI/CD, security, and monitoring** into one product.

1. GitLab Overview

- **Type:** All-in-one DevOps platform
- **Core Strengths:** Built-in **Source Code Management (SCM)** + **CI/CD** + **Security** + **Project Management**
- **Deployment:** SaaS (cloud.gitlab.com) or self-managed (on your own servers)

📌 **Key Idea:** You can go from *idea* → *code* → *test* → *deploy* → *monitor* without leaving GitLab.

2. Comparison with Other Platforms

Feature / Platform	GitLab🦊	GitHub Actions🐱	Jenkins🔧	CircleCI🔄	AWS CodePipeline☁️
SCM (Source Code)	✅ Built-in Git hosting	✅ Built-in Git hosting	❌ Needs GitHub/Bitbucket	❌ Needs GitHub/Bitbucket	❌ Needs GitHub/CodeCommit
CI/CD	✅ Built-in pipelines (YAML-based)	✅ Actions + workflows	✅ Highly customizable	✅ Fast pipelines	✅ AWS service integrations
Hosting	SaaS + self-hosted	SaaS + self-hosted Enterprise	Self-hosted only	SaaS + server option	AWS-only
Ease of Use	👍 Simple UI, all-in-one	👍 Easy if on GitHub	⚠️ Steep learning curve	👍 Simple setup	👍 Easy for AWS users
Integrations	Many built-in	Marketplace Actions	Plugins (huge ecosystem)	API + integrations	AWS services only
Security Features	SAST, DAST, container	Limited(via Actions or 3rd-party)	Needs plugins	Needs 3rd-party tools	IAM-based security

Feature / Platform	GitLab 	GitHub Actions 	Jenkins 	CircleCI 	AWS CodePipeline 
	scans built-in				
Cost	Free tier + paid tiers	Free tier + paid	Free (self-host) + maintenance cost	Paid	Pay per usage
Best For	Teams wanting one tool for everything	GitHub users	Highly customized workflows	Fast cloud CI/CD	AWS-only environments

📖 **Self-Hosted** means **you install and run the software yourself on your own servers (physical or cloud)** instead of using the vendor's cloud service.

📌 In the context of GitLab

- **GitLab SaaS** → You use GitLab's official cloud service (gitlab.com). They handle hosting, updates, backups.
- **GitLab Self-Hosted** → You download GitLab's software and install it on **your own server** (could be AWS EC2, Azure VM, on-premises hardware, etc.).

You are responsible for:

- ⚙️ **Setup & configuration**
- 🔄 **Updates & patches**
- 📁 **Storage & backups**
- 🔒 **Security & access control**
- 💰 **Server cost**

⚡ Analogy

- **Cloud SaaS (GitLab.com)** → Like renting a car 🚗 — you just drive, the company takes care of maintenance.
- **Self-Hosted (GitLab on your server)** → Like owning your car 🚗 — you control everything, but you're responsible for fuel, repairs, and upkeep.

✅ Why choose self-hosted GitLab?

1. **Full control** over data, configuration, and integrations.
2. **Better security & compliance** — useful for industries like banking, healthcare, defense.
3. **Custom integrations** not possible in SaaS.

4. **Unlimited CI/CD minutes** (since you run your own runners).

⚠ **Drawbacks**

- You must manage hardware, scaling, and uptime.
- Requires a **DevOps/sysadmin team** for maintenance.
- If your server goes down, your GitLab is unavailable.

🔗 3. GitLab's CI/CD Strengths

- 📄 **Single YAML File** (`.gitlab-ci.yml`) defines pipeline stages.
- 🛡 **Built-in Security Scans** (SAST, DAST, dependency checks).
- 📄 **Self-hosting Option** for private environments.
- 📊 **Built-in Project Management** (issues, boards, milestones).
- 📦 **Container Registry** built-in (store Docker images).
- 🔄 **Auto DevOps** – automatically detects your app's language & creates pipelines.

⚠ 4. GitLab Limitations

- 🚀 Slower runners compared to CircleCI for large builds.
- 🏢 Fewer marketplace integrations than GitHub Actions.
- 💰 Paid tiers needed for advanced features (e.g., 50+ CI minutes/month on SaaS).

🎯 5. When to Choose GitLab

- ✅ You want **source control + CI/CD + DevSecOps** in one platform.
- ✅ You prefer **self-hosting** for security/compliance.
- ✅ Your team values **built-in security & compliance** tools.
- ✅ You don't want to maintain separate tools like Jenkins + Jira + Nexus.

----GitLab vs GitHub

Both **GitHub** and **GitLab** are powerful platforms for **version control, collaboration, and CI/CD** . While there's a lot of overlap, there are also key differences in philosophy, ecosystem, and use cases.

🔑 Similarities (Both Can Do These)

- **Version Control** → Git-based repo hosting.
- **Collaboration** → Issues, pull/merge requests, code review, wikis, project boards.
- **CI/CD** → Both support pipelines (GitHub Actions / GitLab CI).

- **Security** → Dependency scanning, code quality checks, access control.
- **Open Source & Private Repos** → Both allow hosting public or private repositories.

So yes, you can technically **do everything in GitLab that you can do in GitHub** (and vice versa for most cases).

⚙️ Differences – Why Some Prefer GitHub

1. 🌐 Ecosystem & Community

- GitHub has the **largest open-source community** (millions of public projects).
- Popular for **open-source collaboration**, libraries, and frameworks.
- Example: Most major OSS projects (React, Linux, Node.js, TensorFlow) are on **GitHub**.

2. 🛠️ Integrations & Marketplace

- GitHub has a **huge marketplace** of third-party apps and actions (DevOps, testing, deployments).
- Many SaaS tools **first release GitHub integration** (e.g., Snyk, SonarCloud, AWS Amplify).

3. 🧑‍💻 Developer Familiarity

- GitHub is considered the **default portfolio platform**. Recruiters often ask for a GitHub profile.
- GitHub's UI is slightly simpler for new developers compared to GitLab's enterprise-focused design.

4. ⚡ GitHub Actions vs GitLab CI

- GitHub Actions = Easier to get started, lots of ready-made workflows.
- GitLab CI = More flexible for **complex pipelines**, built deeply into GitLab.

🏢 Why Some Companies Use GitLab Instead

1. 🛠️ All-in-One DevOps Platform

- GitLab integrates **SCM + CI/CD + Issue Tracking + Security + Registry + Monitoring** in one tool.
- GitHub requires third-party apps for some of these.

2. 🔒 Self-Hosting Option

- GitLab can be run **on your own servers** (GitLab Self-Managed).
- Useful for enterprises with **strict data security or compliance needs**.
- GitHub Enterprise also exists, but GitLab's self-hosting is more **mature & flexible**.

3. 📊 Advanced DevOps Features

- GitLab includes **value stream analytics, code quality reports, release orchestration, security dashboards**.
- GitHub is catching up but still behind in some **enterprise DevSecOps** features.

4. 💰 Cost Model

- GitLab often includes features **for free** that GitHub charges for (e.g., unlimited private repos before GitHub made it free, built-in CI minutes).

When to Choose Which?

-  **Use GitHub if**
 - You want to contribute to open-source or build a developer portfolio.
 - You need **huge community support** and easy integrations.
 - You want simplicity and quick setup.
-  **Use GitLab if**
 - You're an **enterprise/team** looking for an **all-in-one DevOps platform** .
 - You want **self-hosting** for compliance/security.
 - You need **advanced CI/CD and DevSecOps features** .

 In short:

- **GitHub = Community + Open Source + Integrations** .
- **GitLab = Enterprise DevOps + Self-Hosting + Full Control** .

----GitLab vs GitHub Actions for CI/CD

Both **GitLab** and **GitHub** are widely used platforms that provide version control, collaboration tools, and CI/CD features. Let's break down the comparison between **GitLab CI/CD** and **GitHub Actions** :

1. Core Concept

- **GitLab CI/CD** :
 - Built-in **CI/CD pipeline engine** .
 - Uses `.gitlab-ci.yml` file for pipeline definitions.
 - Pipelines run on GitLab Runners (can be shared runners or self-hosted).
- **GitHub Actions** :
 - Workflow automation platform integrated into GitHub.
 - Uses `.github/workflows/*.yaml` files.
 - Runs on GitHub-hosted runners (Ubuntu, Windows, macOS) or self-hosted.

2. Configuration

- **GitLab** :
 - `.gitlab-ci.yml` → defines **stages** (build, test, deploy).
 - More **pipeline-focused** with jobs, artifacts, caching, dependencies.
 - Example:

```
stages:
  - build
  - test
  - deploy
```

```
build-job:
  stage: build
  script: echo "Building project"

test-job:
  stage: test
  script: echo "Running tests"
```

- **GitHub Actions :**
- `.github/workflows/main.yml` → defines **jobs & steps** .
- Event-driven (push, pull_request, schedule, etc.).
- Example:

```
name: CI Pipeline
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: echo "Building project"
```

3. Hosting

- **GitLab CI/CD :**
- GitLab.com → Shared runners provided.
- Self-hosted runners possible for custom infra.
- Works with **self-hosted GitLab** (great for private companies).
- **GitHub Actions :**
- Mostly cloud-based runners.
- Supports **self-hosted runners** but less common for enterprise.

4. Features

- **GitLab CI/CD :**
 - ✓ Full **DevOps lifecycle** (issues, planning, repo, CI/CD, monitoring).
 - ✓ **Environments** & review apps.
 - ✓ Advanced caching & artifact handling.
 - ✓ Auto DevOps (pre-built templates for CI/CD).
- **GitHub Actions :**
 - ✓ Huge **marketplace** of reusable actions.
 - ✓ Easy to set up for GitHub repos.

✅ Tight integration with GitHub ecosystem (PR checks, Dependabot, CodeQL).

❌ Limited advanced features compared to GitLab pipelines (though improving).

💰 5. Pricing

- **GitLab :**
- Free tier includes **400 CI/CD minutes/month** on GitLab.com shared runners.
- Paid tiers → more minutes & enterprise features.
- **GitHub Actions :**
- Free tier includes **2,000 CI/CD minutes/month** (for public/private repos).
- Paid based on usage of minutes & storage.

🏢 6. Enterprise Use Cases

- **GitLab CI/CD :**
- Favored by enterprises needing **self-hosted** , **end-to-end DevOps** .
- Strong in **regulated industries** (banks, healthcare) due to compliance.
- **GitHub Actions :**
- Favored by **open-source projects** (GitHub is open-source friendly).
- Easier onboarding for developers already using GitHub repos.

🏆 Summary

- Use **GitLab CI/CD** if:
 - ✅ You want a full DevOps platform (repo + CI/CD + monitoring).
 - ✅ You want **self-hosting & control** .
 - ✅ You need enterprise features like Auto DevOps, advanced security.
- Use **GitHub Actions** if:
 - ✅ Your repos are already on GitHub.
 - ✅ You want **simple, event-driven workflows** .
 - ✅ You want access to a massive **marketplace of prebuilt actions** .

👉 In short:

- **GitLab = All-in-one DevOps platform.**
- **GitHub Actions = Flexible CI/CD inside GitHub ecosystem.**

----GitLab Pipeline

◆ What is a GitLab Pipeline?

A **GitLab Pipeline** is an automated process that runs in stages to **build, test, and deploy** your code whenever changes are pushed to the repository.

Think of it like an assembly line: each stage has jobs, and jobs can run in parallel or sequentially depending on dependencies.

push code → GitLab Runner
↓
pipeline
↓
jobs

◆ GitLab CI/CD Flow

1. **Push code** → to GitLab repo.
2. **GitLab Runner** picks up jobs defined in `.gitlab-ci.yml`.
3. **Pipeline** runs with multiple **stages** (build, test, deploy).
4. Each **job** executes inside a runner (e.g., Docker container, shell, Kubernetes pod).
5. Results are reported back to GitLab UI.

◆ Key Components of GitLab Pipelines

1. **Pipeline** → The full execution of all stages.
2. **Stages** → Ordered steps in the pipeline (e.g., `build` → `test` → `deploy`).
3. **Jobs** → Actual tasks inside a stage (e.g., "run unit tests").
4. **Runners** → Agents that execute jobs (can be GitLab shared runners or self-hosted).
5. **Artifacts** → Files produced by jobs (e.g., build output, logs).
6. **Environments** → Deployment targets (e.g., staging, production).

◆ Example `.gitlab-ci.yml`

Here's a simple pipeline:

```
stages:                # Define the order of stages
  - build
  - test
  - deploy

build-job:              # Job 1
  stage: build
  script:
    - echo "Compiling the code..."
    - gcc main.c -o main

test-job:               # Job 2
  stage: test
  script:
    - echo "Running tests..."
    - ./main --test

deploy-job:             # Job 3
  stage: deploy
  script:
    - echo "Deploying to server..."
    - scp main user@server:/app/
```

only:

- main

Deploy only when code is pushed to main branch

◆ How Pipelines Run

- **Stages run sequentially** : build → test → deploy.
- **Jobs in the same stage run in parallel** .
- If any job fails , the pipeline stops (unless configured otherwise).
- **Branch rules** can decide when jobs run (e.g., run deploy only on main).

◆ Types of GitLab Pipelines

1. **Basic pipeline** → sequential stages.
2. **Multi-project pipeline** → triggers pipelines across projects.
3. **Parent-child pipeline** → a pipeline that triggers sub-pipelines.
4. **Merge request pipelines** → run only when a MR is created/updated.
5. **Scheduled pipelines** → run at specific times (cron jobs).
6. **Manual pipelines** → triggered by developers.

- Basic
- Multi-project
- Merge Request
- Parent-child
- Scheduled
- Manual.

◆ Steps to Create a Pipeline in GitLab GUI

1. Go to Your Project

- Open your project in GitLab.
- From the left sidebar, go to **Build** → **Pipelines** .

2. Click “Run Pipeline”

- You’ll see a button **Run Pipeline** at the top right.
- This lets you run a pipeline manually (you can select branch and variables before running).

3. CI/CD Editor (GUI for .gitlab-ci.yml)

- If you don’t want to write YAML by hand, GitLab has a **Pipeline Editor** :
 - Navigate to **CI/CD** → **Editor** .
 - This opens a GUI/YAML editor.
 - GitLab gives you **templates** (for Node.js, Python, Java, Docker, etc.).
 - You can visually configure jobs, stages, runners, and variables.

4. Pipeline Wizard (Pipeline Editor → Visual Mode)

- Inside the **Pipeline Editor** , there’s a **Visualize** tab.
- It shows a **graph view** of your pipeline stages and jobs.
- This makes it easier to confirm the workflow.

5. Add Jobs in GUI

- Example: If you want a build + test + deploy pipeline:
 - Stage: **build** → Add script like **npm install**.
 - Stage: **test** → Add script like **npm test**.
 - Stage: **deploy** → Add script for deployment.

- You can add these jobs in the editor without directly writing YAML (GitLab will generate the `.gitlab-ci.yml` in the repo).

6. Validate Pipeline

- GitLab automatically validates the pipeline before committing.
- You can also go to **CI/CD** → **Lint** to paste YAML or check the generated one.

7. Commit & Run

- Once satisfied, commit changes.
- GitLab will trigger a pipeline automatically, or you can **manually trigger** from the Pipelines page.

Example Using GUI (Generated YAML)

If you set up a Node.js build + test pipeline in GUI, GitLab will create something like:

```
stages:
  - build
  - test

build-job:
  stage: build
  script:
    - npm install

test-job:
  stage: test
  script:
    - npm test
```

You don't need to write this manually—GitLab generates it from GUI.

✅ So, summary :

- **Run Pipeline button** → For manual runs.
- **Pipeline Editor (GUI + Templates + Visualizer)** → To create and edit pipelines without deep YAML knowledge.
- GitLab still saves config as `.gitlab-ci.yml` in repo, but GUI helps beginners.

◆ Benefits of GitLab Pipelines

- ✅ Automates **CI/CD** (no manual builds/deployments).
- ✅ Works seamlessly with **GitLab issues, merge requests, and repos** .
- ✅ Supports **Docker, Kubernetes, and cloud deployments** .
- ✅ Provides **pipeline visualization** in GitLab UI.
- ✅ Fully **customizable** via `.gitlab-ci.yml`.

👉 In short:

- A **pipeline** = the full process.
- A **stage** = a step in the pipeline.
- A **job** = a task inside a stage.
- A **runner** = the executor of jobs.

----GitLab Runners (For more Info see -- Keyword - tags)

◆ What are GitLab Runners?

A **GitLab Runner** is an agent (process/application) that runs jobs defined in your `.gitlab-ci.yml` file.

- They are responsible for executing your pipeline jobs (like build, test, deploy).
- Without a runner, pipelines won't run, even if you configure `.gitlab-ci.yml`.

Think of them as **workers** waiting for jobs from GitLab.

◆ Types of GitLab Runners

There are two main types:

1. Shared Runners

- Available to all projects in a GitLab instance.
- Maintained by the GitLab admin (or GitLab.com in SaaS).
- Example: GitLab.com provides free shared runners (though limited in free tier).

2. Specific Runners

- Dedicated to one or more specific projects/groups.
- Useful when you need **custom environments**, like special dependencies, larger compute power, or secure networks.

◆ Runner Executors

Server → OS → Runner → Executor

Runners use **executors** to decide how and where to run jobs. Runners are installed inside a server of any OS and inside that runners, executor is present

Some common executors are:

- ✓ **Shell Executor** → Runs jobs directly on the machine's shell.
- ✓ **Docker Executor** → Runs each job inside a Docker container. Default Docker image is **RUBY**
 - **Docker Machine Executor** → Auto-creates Docker machines on-demand.
- ✓ **Kubernetes Executor** → Runs jobs as Kubernetes Pods.

- **VirtualBox / Parallels** → For VMs.

👉 Most popular in real-world projects: **Docker & Kubernetes** .

◆ How Runners Work (Flow)

1. You push code → triggers pipeline.
2. GitLab schedules jobs from `.gitlab-ci.yml`.
3. Runner polls GitLab for jobs (pull model).
4. Runner picks up a job → prepares environment (Docker, shell, etc.).
5. Executes steps (e.g., install deps, run tests, deploy).
6. Sends logs & results back to GitLab.

◆ Installing a GitLab Runner

For self-hosted setups, you install runners manually. Example for Linux:

```
# Download the runner
sudo apt-get install gitlab-runner

# Register the runner
gitlab-runner register
```

During registration, you'll provide:

- **URL** of GitLab instance
- **Registration token** (from GitLab project/group)
- **Executor type** (Docker, shell, etc.)
- **Tags** (to target jobs in `.gitlab-ci.yml`)

IMT, used later to specify the runners

◆ Example Runner Configuration

File: `/etc/gitlab-runner/config.toml`

```
[[runners]]
  name = "docker-runner"
  url = "https://gitlab.com/"
  token = "YOUR_RUNNER_TOKEN"
  executor = "docker"
  [runners.docker]
    image = "node:18"
    privileged = true
    volumes = ["/cache"]
```

◆ Using Tags to Target Runners

In `.gitlab-ci.yml`:

```
build_job:
  stage: build
  script:
    - echo "Building app"
  tags:
    - docker-runner
```

👉 This ensures only a runner with tag `docker-runner` will pick up this job.

◆ Key Benefits of Custom Runners

- Full control over environment.
- Can use private networks/resources (e.g., internal databases).
- Run heavier workloads than GitLab shared runners.
- Security: no risk of other orgs using the same runner machine.

✅ In short :

- **Runners = workers** that execute your CI/CD jobs.
- They can be **shared** (GitLab-wide) or **specific** (dedicated).
- Support multiple **executors** (Shell, Docker, Kubernetes).
- Installed & configured via registration tokens.

----Multiple Jobs

Multiple jobs without writing `stages` key will run parallelly --

The image shows a video player interface with a GitLab CI/CD pipeline configuration. The video title is "GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]". The video progress bar shows 21:53 / 1:14:33. The video content displays a GitLab CI/CD pipeline configuration for a project named "Test". The pipeline has three jobs: "build-job", "test-job", and "deploy-job". The "build-job" is configured with a stage of "build" and a script that echoes "First step" and "Second Step". The "test-job" is configured with a script that echoes "Testing Step". The "deploy-job" is configured with a script that echoes "Deploy Step..". The video player interface includes a sidebar with navigation links (Project, Test, Learn GitLab, Pinned, Issues, Merge requests, Manage, Plan, Code, Build, Secure, Deploy, Operate, Monitor, Analyze, Settings) and a main content area showing the pipeline configuration. A circular inset in the bottom right corner shows a person speaking into a microphone.

```
build-job:
  stage: build
  script:
    - echo "First step"
    - echo "Second Step"

test-job:
  script:
    - echo "Testing Step"

deploy-job:
  script:
    - echo "Deploy Step.."
```

GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]

Test / Test / Pipelines / #1296247522

Update .gitlab-ci.yml file

Running paulphilip1 created pipeline for commit 1cb846e7 just now

For main
latest 3 jobs In progress, queued for 0 seconds

Pipeline Needs Jobs 3 Tests 0

test

- build-job
- deploy-job
- test-job

Cancel pipeline Delete

21:31 / 1:14:33 • Multiple Jobs >

To make it run sequentially, use **stages** key --

GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]

Test / Test / Pipeline Editor

Browse CI/CD Catalog Browse templates Help

```
1 stages: I
2   - build
3   - test
4   - deploy
5
6 #Build step
7 build-job:
8   stage: build
9   script:
10     - echo "First step"
11     - echo "Second Step"
12
13 test-job:
14   stage: test
15   script:
16     - echo "Testing Step"
17
18 deploy-job:
19   stage: deploy
20   script:
21     - echo "Deploy Step.."
22
```

23:01 / 1:14:33 • Multiple Jobs >

GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]

Test / Test / Pipelines / #1296248443

Update .gitlab-ci.yml file

Running paulphilip1 created pipeline for commit c1bcc3be just now

For main

latest 3 jobs In progress, queued for 0 seconds

Pipeline Needs Jobs 3 Tests 0

build test deploy

build-job test-job deploy-job

23:18 / 1:14:33 • Multiple Jobs >

To see the Job logs--

GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]

Test / Test / Pipelines / #1296248443

Update .gitlab-ci.yml file

Passed paulphilip1 created pipeline for commit c1bcc3be 13 minutes ago, finished 12 minutes ago

For main

latest 3 jobs 1.4 1 minute 23 seconds, queued for 0 seconds

Pipeline Needs Jobs 3 Tests 0

build test deploy

build-job test-job deploy-job

24:43 / 1:14:33 • Multiple Jobs >

GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]

Test / Test / Jobs / #6885218556

build-job

✓ Passed Started 14 minutes ago by paulphilip1

Search job log

```

1 Running with gitlab-runner 17.0.0-pre.88.g761ae5dd (761ae5dd)
2 on green-4.saas-linux-small-amd64.runners-manager.gitlab.com/default ntfFetyX, system I
  D: s_8998de21c559
3 Preparing the "docker+machine" executor 00:19
4 Using Docker executor with image ruby:3.1 ...
5 Pulling docker image ruby:3.1 ...
6 Using docker image sha256:94b52c1182e066a3ccf48bddaae3bdac7265ab6d784d53a0c6f6486e9a49463
  6 for ruby:3.1 with digest ruby@sha256:0a72949c3f03c00bb9c63b382db578d2ecb743a288d59c3a3
  22845529b510a42 ...
7 Preparing environment 00:05
8 Running on runner-ntfetyx-project-58000654-concurrent-0 via runner-ntfetyx-s-l-s-amd64-
  1716065422-bb21ca8d...
9 Getting source from Git repository 00:01
10 Fetching changes with git depth set to 20...
11 Initialized empty Git repository in /builds/test6912220/Test/.git/
12 Created fresh repository.
13 Checking out c1bcc3be as detached HEAD (ref is main)...
14 Skipping Git submodules setup
15 $ git remote set-url origin "${CI_REPOSITORY_URL}"
16 Executing "step_script" stage of the job script 00:00
17 Using docker image sha256:94b52c1182e066a3ccf48bddaae3bdac7265ab6d784d53a0c6f6486e9a49463
  6 for ruby:3.1 with digest ruby@sha256:0a72949c3f03c00bb9c63b382db578d2ecb743a288d59c3a3
  22845529b510a42 ...
18 $ echo "First step"
19 First step

```

Duration: 27 seconds
 Finished: 14 minutes ago
 Queued: 0 seconds
 Timeout: 1h (from project)
 Runner: #12270857 (ntfFetyXQ) 4-green.saas-linux-small-amd64.runners-manager.gitlab.com/default
 Commit c1bcc3be
 Update .gitlab-ci.yml file
 Pipeline #1296248443 ✓ Passed for main
 build

Related jobs
 → ✓ build

24:50 / 1:14:33 • Multiple Jobs > Second Step

1. Basic Structure of Multiple Jobs

```

stages:
  - build
  - test
  - deploy

```

```
# Job 1
```

```
build-job:
```

```
  stage: build
```

```
  script:
```

- echo "Compiling code..."
- echo "Build complete!"

```
# Job 2
```

```
test-job:
```

```
  stage: test
```

```
  script:
```

- echo "Running tests..."
- echo "Tests passed!"

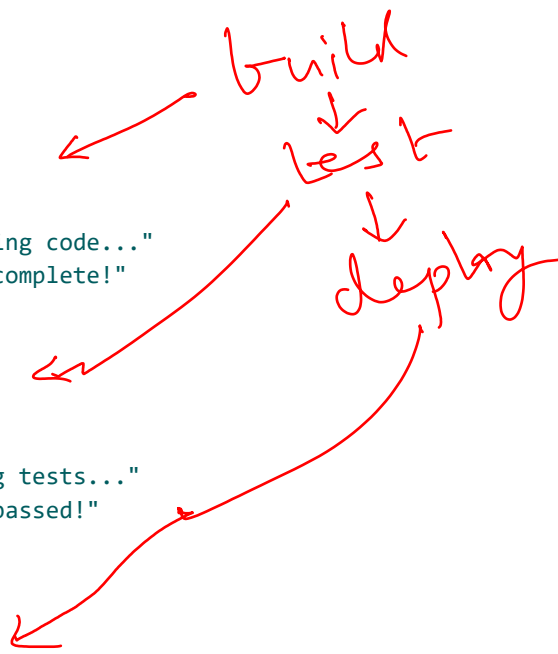
```
# Job 3
```

```
deploy-job:
```

```
  stage: deploy
```

```
  script:
```

- echo "Deploying application..."
- echo "Deployment done!"



◆ How this runs:

1. **build-job** runs first (since it's in the **build** stage).
2. **test-job** runs after **build-job** (since it's in the **test** stage).

3. **deploy-job runs last** (since it's in the **deploy** stage).

👉 This is **sequential execution** because stages run in order.

2. Multiple Jobs Running in Parallel

If you have **two or more jobs in the same stage** , they run in **parallel** .

```
stages:
  - test
  - deploy

# Job 1
unit-tests:
  stage: test
  script:
    - echo "Running unit tests..."

# Job 2
integration-tests:
  stage: test
  script:
    - echo "Running integration tests..."

# Job 3
deploy-job:
  stage: deploy
  script:
    - echo "Deploying app..."
```

◆ How this runs:

- **unit-tests** and **integration-tests** run at the same time (since both belong to the **test** stage).
- Once **both finish** , **deploy-job** starts.

3. Mix of Sequential + Parallel

You can combine both:

```
stages:
  - build
  - test
  - deploy

# Sequential: runs first
build-job:
  stage: build
  script:
    - echo "Building project..."

# Parallel: both run together after build-job
unit-tests:
```

```
stage: test
script:
  - echo "Running unit tests..."
```

```
lint-check:
  stage: test
  script:
    - echo "Running lint checks..."
```

```
# Sequential: runs after test stage jobs finish
deploy-job:
  stage: deploy
  script:
    - echo "Deploying app..."
```

◆ Flow:

1. **build-job** runs → finishes.
2. **unit-tests** + **lint-check** run **together** .
3. **deploy-job** runs after both test jobs finish.

✓ Summary :

- **Stages define order** (sequential between stages).
- **Jobs inside the same stage run in parallel** .
- You control flow with **stages**.

----Keyword - **before_script** and **after_script**

◆ **before_script**

- **Definition** : A list of commands that run **before each job's script section** (unless overridden).
- You can define it globally (applies to all jobs) or at the job level.
- Common use: setup tasks like installing dependencies, configuring environment, logging in to registries.

✓ Example:

```
default:
  before_script:
    - echo "This runs before every job"
    - apt-get update -y

job1:
  script:
    - echo "Running Job 1"

job2:
  before_script:
```



```
- echo "Custom before_script only for job2"
script:
- echo "Running Job 2"
```

👉 In this case:

- **job1** runs both global **before_script** commands.
- **job2** overrides and runs only its own **before_script**.

◆ **after_script**

- **Definition** : A list of commands that run **after the script section** , regardless of job success or failure.
- Useful for **cleanup tasks** (removing temp files, logging, stopping services, etc.).

✅ Example:

```
job_with_after_script:
  script:
    - echo "Main job script"
    - exit 1 # simulate failure
  after_script:
    - echo "This still runs after the job"
    - rm -rf temp/
```

👉 Even if the job fails (**exit 1**), the **after_script** still runs.

◆ **Combined Example with Multiple Jobs**

```
default:
  before_script:
    - echo "Global setup runs before all jobs"

job1:
  script:
    - echo "Job 1 running"

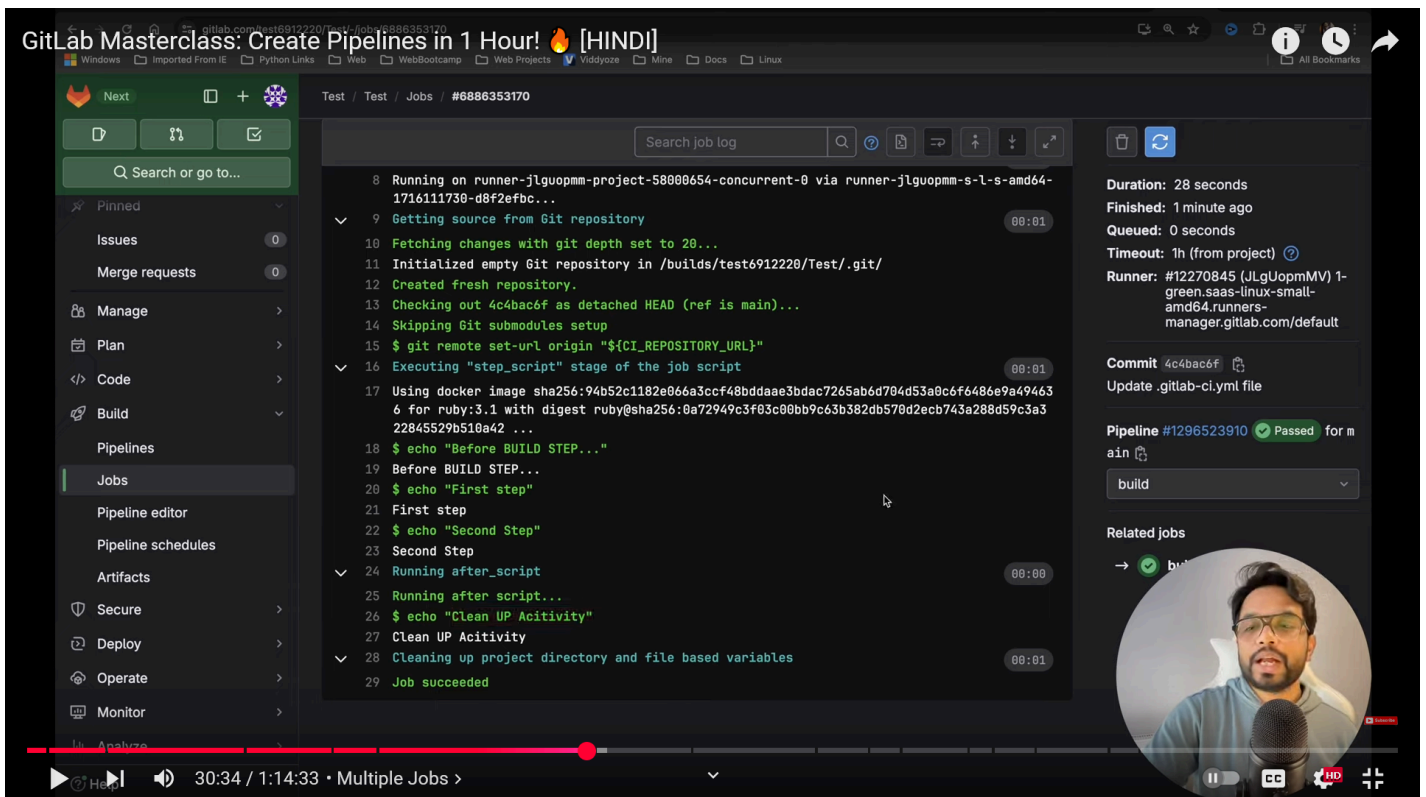
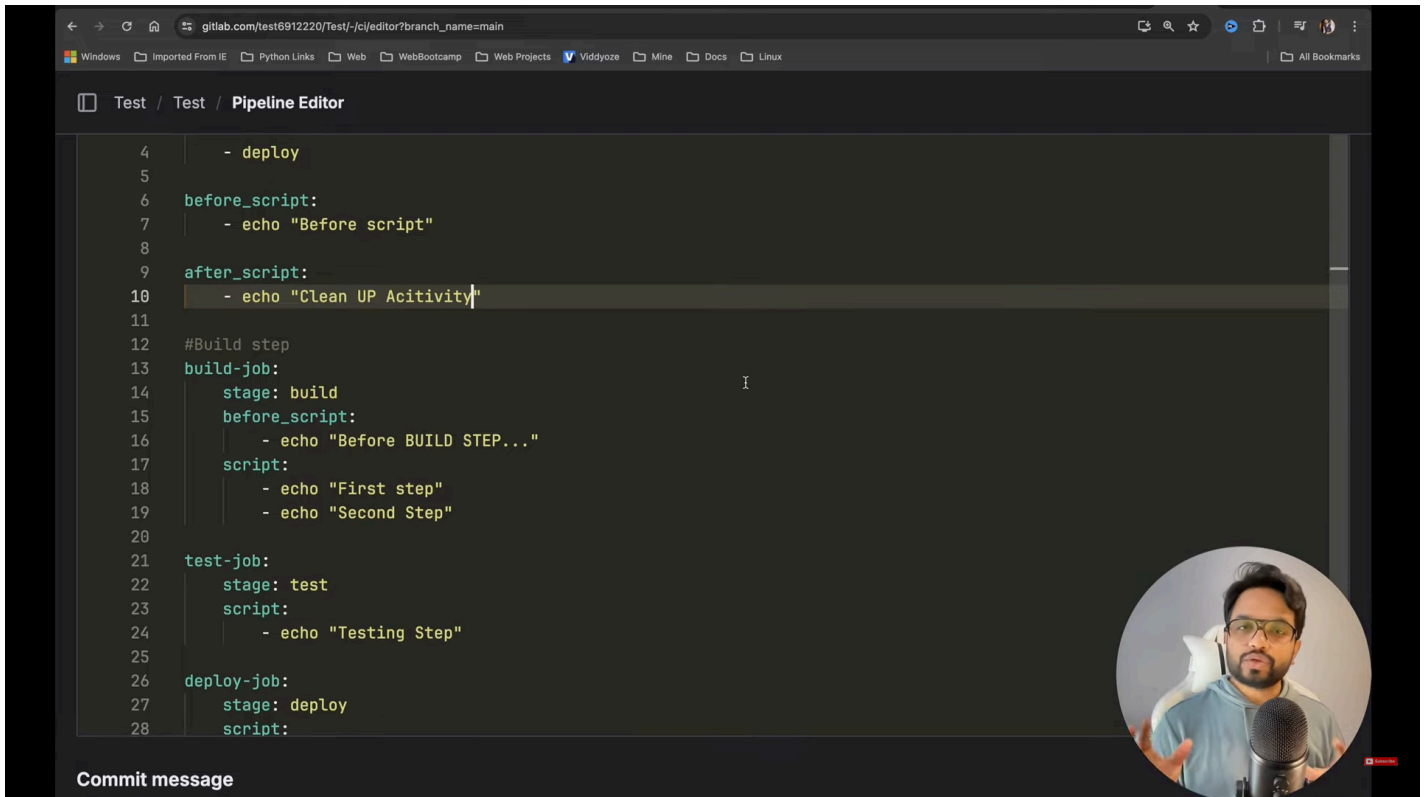
job2:
  script:
    - echo "Job 2 running"
  after_script:
    - echo "Cleaning up after Job 2"
```

Execution order for job2:

1. Run global **before_script** (echo "Global setup...")
2. Run **script** (echo "Job 2 running")
3. Run **after_script** (echo "Cleaning up...")

🔑 **Key Points**

- **before_script**: Pre-job setup.
- **script**: Main job execution.
- **after_script**: Post-job cleanup (runs even on failure).
- You can override **before_script/after_script** at the job level.



----GitLab Artifacts

◆ What are GitLab Artifacts?

In GitLab CI/CD, **artifacts** are files or directories created by a job that you want to preserve after the job finishes.

They are usually things like:

- Compiled binaries
- Test reports
- Log files
- Build outputs (e.g., `.zip`, `.jar`, `.war` files)
- Coverage reports

Artifacts can be:

- **Downloaded** manually from the GitLab UI.
- **Shared** with downstream jobs in the same pipeline.

Without artifacts, once a job finishes, all files generated inside its runner's environment are gone.

◆ Defining Artifacts in `.gitlab-ci.yml`

You define artifacts inside a job using the `artifacts` keyword.



Example:

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - echo "Compiling app..."
    - mkdir build
    - echo "binary-data" > build/app.bin
  artifacts:
    paths:
      - build/app.bin
    expire_in: 1 week # (Optional) Expiration time

test_job:
  stage: test
  script:
    - echo "Running tests..."
    - cat build/app.bin
  dependencies:
    - build_job # Gets artifacts from build_job
```

🔍 Breakdown:

- `artifacts:`

- **paths:** → Files/directories to save as artifacts.
- **expire_in:** → How long artifacts are kept (default: forever).
- **when:** → Defines when to upload artifacts (**on_success**, **on_failure**, **always**).
- **reports:** → Special type for test/coverage/security reports.

◆ Artifact Expiration

- Default: **forever** (takes storage).
- You can set expiration:

```
artifacts:
  paths:
    - build/
  expire_in: 3 days
```

After 3 days, GitLab automatically deletes them.

◆ Artifact Passing Between Jobs

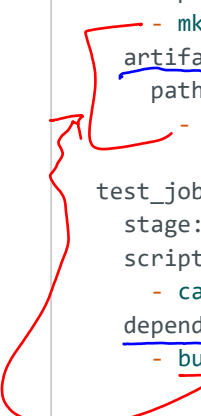
Artifacts from one job can be **used in another job** (but only in later stages).

You need **dependencies** or **needs** keyword.

Example: Sequential passing

```
build_job:
  stage: build
  script:
    - mkdir dist && echo "app" > dist/app.txt
  artifacts:
    paths:
      - dist/

test_job:
  stage: test
  script:
    - cat dist/app.txt
  dependencies:
    - build_job
```



Here, **test_job** gets the **dist/app.txt** artifact from **build_job**.

◆ Special Reports (Test, Coverage, etc.)

Artifacts can also be **parsed by GitLab UI** if declared as reports.

Example (JUnit Test Report):

```
test_job:
  stage: test
  script:
    - run-tests.sh --junit-report=report.xml
  artifacts:
    reports:
      junit: report.xml
```

- GitLab will show test results in **CI/CD** → **Jobs** → **Test Report tab** .

◆ Downloading Artifacts

- Go to **GitLab UI** → **CI/CD** → **Jobs** → **Artifacts** and download.
- Or use GitLab API to fetch them.

✅ Summary :

- **Artifacts = Job outputs saved for later use** .
- Can be downloaded, passed to later jobs, or parsed as reports.
- You control **paths**, **expire_in**, **when**, and **reports**.

----Environmental Variables in GitLab

In GitLab, **environment variables** let you pass dynamic values (like secrets, API keys, paths, or config settings) into your CI/CD pipelines. You can define them at **different levels** : GitLab UI, **.gitlab-ci.yml**, or directly in a job.

◆ 1. Adding Environment Variables in GitLab UI (Recommended for Secrets)

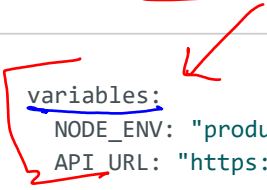
1. Go to your **GitLab project** .
2. Navigate to **Settings** → **CI/CD** .
3. Expand the **Variables** section.
4. Click “**Add variable**” .
 - **Key** → Name of the variable (e.g., **AWS_SECRET_KEY**).
 - **Value** → Secret value.
 - **Flags** :
 - **Protected** → Only available for protected branches/tags.
 - **Masked** → Value won't be exposed in logs.
 - **Environment scope** → Limit to specific environments (e.g., **production**).

Now these variables are available in all jobs of your pipeline.

◆ 2. Adding Variables in **.gitlab-ci.yml**

You can define **global variables** or **job-specific variables** .

Example – Global Variables



```
variables:
  NODE_ENV: "production"
  API_URL: "https://api.example.com"

stages:
  - build
  - deploy

build-job:
  stage: build
  script:
    - echo "Building for $NODE_ENV"

deploy-job:
  stage: deploy
  script:
    - echo "Deploying to $API_URL"
```

Example – Job-Specific Variables

```
deploy-job:
  stage: deploy
  variables:
    DEPLOY_ENV: "staging"
  script:
    - echo "Deploying to $DEPLOY_ENV"
```

◆ 3. Passing Variables at Runtime (Trigger or Schedule)

When you trigger a pipeline manually:

- Go to **CI/CD** → **Pipelines** → **Run Pipeline** .
- You can **add key-value pairs** for variables at runtime.

◆ 4. Using Variables in Scripts

Inside a job, variables are accessible as environment variables:

```
test-job:
  stage: test
  script:
    - echo "Node environment is $NODE_ENV"
    - echo "Secret token is $SECRET_TOKEN"
```

gitlab.com/test6912220/Test

Windows Imported From IE Python Links Web WebBootcamp Web Projects Viddyoze Mine Docs Linux

Test / Test

Your account is authenticated with SSO or SAML. To push and pull over HTTPS with Git using this account, you must set a password or set up a Personal Access Token to use instead of a password. For more information, see Clone with HTTPS.

Remind later Don't show again

Project

T Test

Learn GitLab 25%

Pinned

Issues 0

Merge requests 0

Manage

Plan

Code

Build

Secure

Deploy

Operate

Monitor

Analyze

Settings

Help

Test

main Test / +

History Find file Edit Code

Project information

17 Commits

1 Branch

0 Tags

73 KiB Project Storage

README

CI/CD configuration

Add LIC

Add

Add

Add

Add

Config

Created on

General

Integrations

Webhooks

Access Tokens

Repository

Merge requests

CI/CD

Packages and registries

Monitor

Analytics

Usage Quotas

Last commit

Last update

Update .gitlab-ci.yml file 15 minutes ago

Initial commit 16 hours ago

Update basic.sh just now

Getting started

To make it easy for you to get started with GitLab, here's a list of recommended next steps:

gitlab.com/test6912220/Test/-/settings/ci_cd

Windows Imported From IE Python Links Web WebBootcamp Web Projects Viddyoze Mine Docs Linux

Test / Test / CI/CD Settings

A job artifact is an archive of files and directories saved by a job when it finishes.

Variables

Variables store information that you can use in job scripts. Each project can define a maximum of 8000 variables. [Learn more.](#)

Variables can be accidentally exposed in a job log, or maliciously sent to a third party server. The masked variable feature can help reduce the risk of accidentally exposing variable values, but is not a guaranteed method to prevent malicious users from accessing variables. [How can I make my variables more secure?](#)

Variables can have several attributes. [Learn more.](#)

- Protected: Only exposed to protected branches or protected tags.
- Masked: Hidden in job logs. Must match masking requirements.
- Expanded: Variables with \$ will be treated as the start of a reference to another variable.

CI/CD Variables </> 0

Add variable

Key	Value	Environments	Actions
There are no variables yet.			

Group variables (inherited)

These variables are inherited from the parent group.

CI/CD Variables </> 0

Key	Attributes	Environments	Group
There are no variables yet.			

Help

gitlab.com/test6912220/Test/-/settings/ci_cd

WindowsImported From IEPython LinksWebWebBootcampWeb ProjectsViddyozeMineDocsLinux

Secure>

Deploy>

Operate>

Monitor>

Analyze>

Settings>

General

Integrations

Webhooks

Access Tokens

Repository

Merge requests

CI/CD

Packages and registries

Monitor

Analytics

Usage Quotas

Help

Test / Test / CI/CD Settings

Variables can have several attributes. [Learn more.](#)

Protected: Only exposed to protected branches or protected tags.

Masked: Hidden in job logs. Must match masking requirements.

Expanded: Variables with \$ will be treated as the start of a reference to another variable.

CI/CD Variables </> 0

Key ↑	Value	Environment
There are no variables yet.		

Group variables (inherited)

These variables are inherited from the parent group.

CI/CD Variables </> 0

Key	Attributes	Environments
There are no variables yet.		

Pipeline trigger tokens

Trigger a pipeline for a branch or tag by generating a trigger token and using it with an API call. The token and permissions. [Learn more.](#)

Deploy freezes

All (default)

Visibility

Visible

Can be seen in job logs.

Masked

Masked in job logs but value can be revealed in CI/CD settings. Requires values to meet regular expressions requirements.

Flags

Protect variable

Export variable to pipelines running on protected branches and tags only.

Expand variable reference

\$ will be treated as the start of a reference to another variable.

Description (optional)

The description of the variable's value or usage.

Key

NAME

You can use CI/CD variables in different places, but they are not the same. [What is the difference?](#)

Value

GITLAB

Add variable

Cancel

gitlab.com/test6912220/Test/-/settings/ci_cd

Windows Imported From IE Python Links Web WebBootcamp Web Projects Viddyoze Mine Docs Linux

Secure Deploy Operate Monitor Analyze Settings CI/CD Packages and registries Monitor Analytics Usage Quotas Help

Test / Test / CI/CD Settings

Variables can be accidentally exposed in a job log, or maliciously sent to a third party server. The masked risk of accidentally exposing variable values, but is not a guaranteed method to prevent malicious users from making my variables more secure?

Variables can have several attributes. [Learn more.](#)

- Protected: Only exposed to protected branches or protected tags.
- Masked: Hidden in job logs. Must match masking requirements.
- Expanded: Variables with \$ will be treated as the start of a reference to another variable.

CI/CD Variables </> 1

Key ↑	Value	Environment
NAME	*****	All (default)

Protected Expanded

Group variables (inherited)

These variables are inherited from the parent group.

CI/CD Variables </> 0

Key	Attributes	Environments
There are no variables yet.		

Pipeline trigger tokens

Trigger a pipeline for a branch or tag by generating a trigger token and using it with an API call. The token and permissions. [Learn more.](#)

Visibility

☐ Visible
Can be seen in job logs.

☒ Masked
Masked in job logs but value can be revealed in CI/CD settings. Requires values to meet regular expressions requirements.

Flags

☒ Protect variable
Export variable to pipelines running on protected branches and tags only.

☒ Expand variable reference
\$ will be treated as the start of a reference to another variable.

Description (optional)

The description of the variable's value or usage.

Key

PASS

You can use CI/CD variables in different places, but they are not the same. [What is the difference?](#)

Value

TEST1234

Add variable Cancel



GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]

Windows Imported From IE Python Links Web WebBootcamp Web Projects Viddyoze Mine Docs Linux

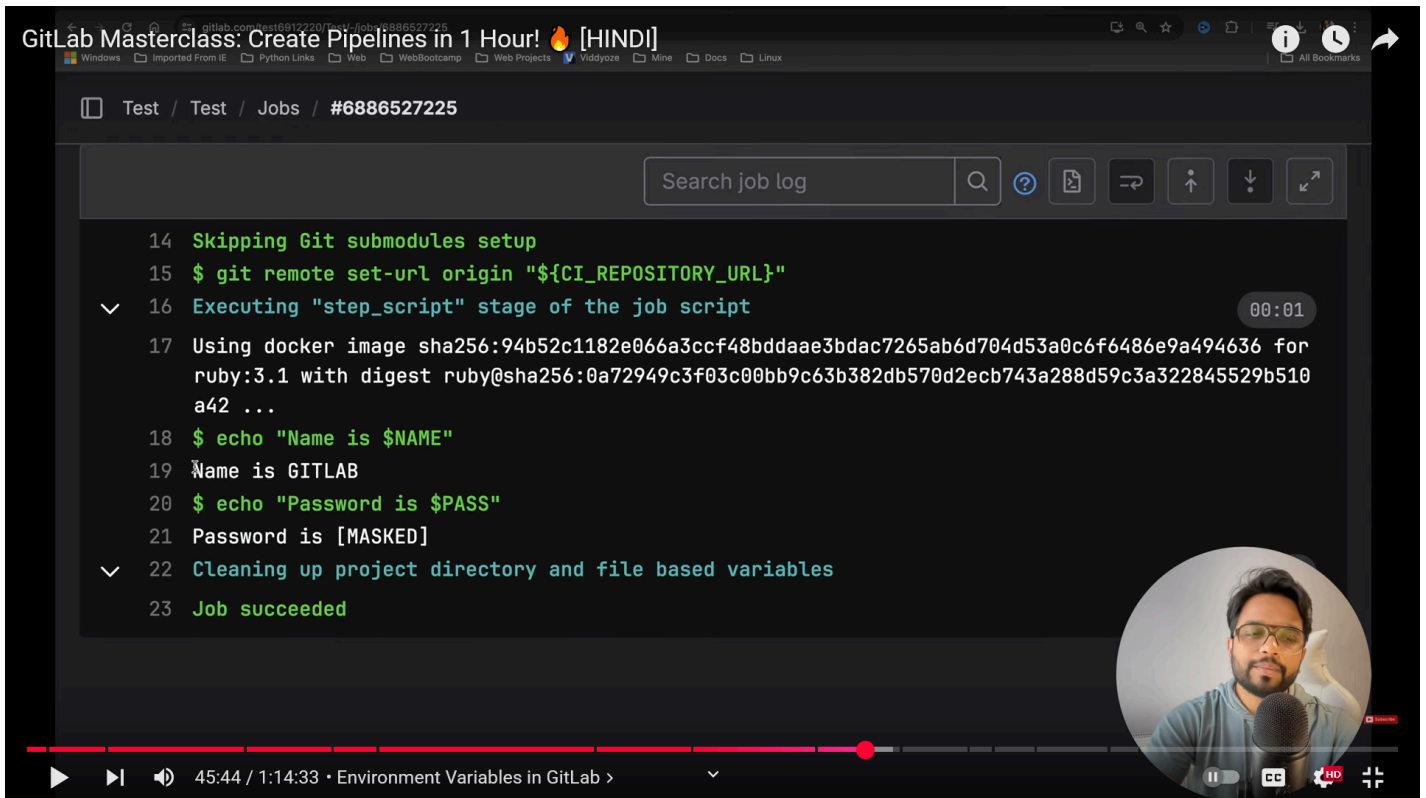
Test / Test / Pipeline Editor

Browse CI/CD Catalog Browse templates Help

```
1 #To run bash script
2
3 bash_execute:
4   script:
5     - echo "Name is $NAME"
6     - echo "Password is $PASS"
```

45:30 / 1:14:33 • Environment Variables in GitLab >





✓ Summary :

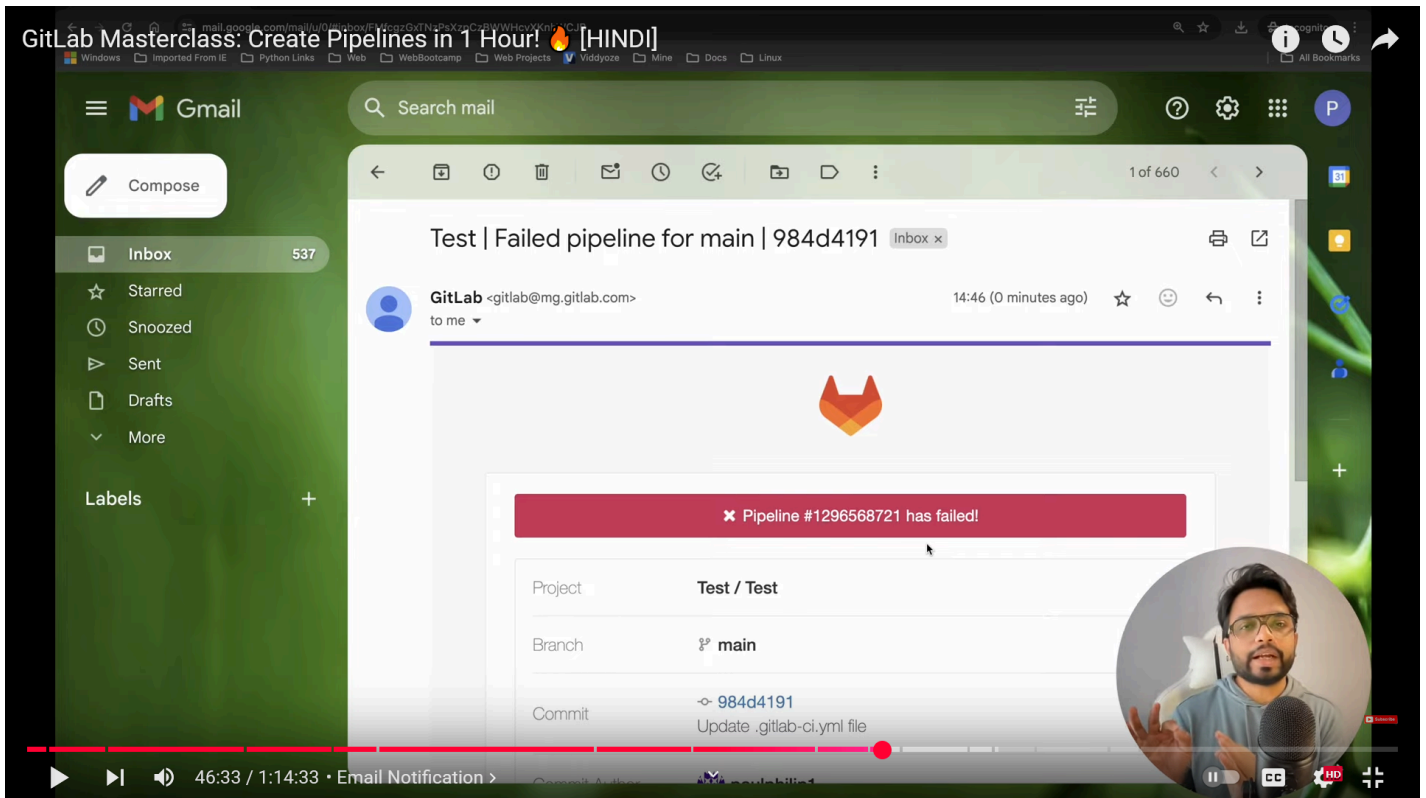
- UI Variables → Best for secrets (hidden & masked).
- `.gitlab-ci.yml` Variables → Best for config values.
- Runtime Variables → Best for flexibility when triggering pipelines manually.

----Email Notifications in GitLab

GitLab supports **email notifications** to keep you updated on activity in your repositories, merge requests, pipelines, and issues.

◆ How Email Notifications Work in GitLab

- Each GitLab user has an **email address** associated with their account.
- GitLab sends email alerts when:
 - Someone assigns you to an issue or merge request.
 - Someone mentions you (`@username`) in a comment.
 - A pipeline/job succeeds or fails (if configured).
 - You subscribe/watch a project or group.



◆ Configuring Email Notifications

There are **two levels** :

1. Personal Notification Settings

Each user can configure how often they receive notifications:

- Go to **Profile** → **Preferences** → **Notifications** .
- Select a level:
 - **Disabled** → No emails.
 - **Participate** → Only when involved (e.g., assigned, mentioned).
 - **Watch** → All activity in the project/group.
 - **Global** → Applies across all projects unless overridden.
 - **Custom** → Choose exactly which events trigger emails.

👉 Example: You can set **Custom** to get emails only for **pipeline failures** but not for comments.

2. Project/Group Level Notifications

- Inside a project → **Settings** → **Notifications** .
- You can override global settings per project.

Example: For an important repo, set to **Watch** , but for others keep it at **Participate** .

◆ Pipeline/Job Email Notifications

You can also explicitly configure **email alerts for CI/CD pipelines** in `.gitlab-ci.yml`.

Example:

```
stages:
  - build
  - test

build-job:
  stage: build
  script:
    - echo "Building..."
  after_script:
    - echo "Build completed"

test-job:
  stage: test
  script:
    - echo "Running tests"
  after_script:
    - echo "Tests finished"
  when: on_failure
  # Add email notification on failure
  allow_failure: false
```

GitLab doesn't have a built-in `email:` keyword in CI (unlike Jenkins), but you can send emails using:

- `mail` or `sendmail` command in scripts.
- GitLab integrations (e.g., SMTP setup).
- Notifications via **Slack/MS Teams** if emails aren't enough.

◆ Admin Setup (SMTP)

For GitLab to send emails:

1. Admin must configure **SMTP settings** in `gitlab.rb`.

```
gitlab_rails['smtp_enable'] = true
gitlab_rails['smtp_address'] = "smtp.gmail.com"
gitlab_rails['smtp_port'] = 587
gitlab_rails['smtp_user_name'] = "your-email@gmail.com"
gitlab_rails['smtp_password'] = "your-password"
gitlab_rails['smtp_domain'] = "gmail.com"
gitlab_rails['smtp_authentication'] = "login"
gitlab_rails['smtp_enable_starttls_auto'] = true
```

2. Reconfigure GitLab:

```
sudo gitlab-ctl reconfigure
```

3. Test email from **Admin Area** → **Monitoring** → **Check Email** .

- Users manage their own **notification preferences** .
- Projects can override notification levels.
- Pipelines can send emails manually via scripts or integrations.
- GitLab admins must configure **SMTP** for the system to send mail.

----Scheduled Pipeline

◆ What is a Scheduled Pipeline in GitLab?

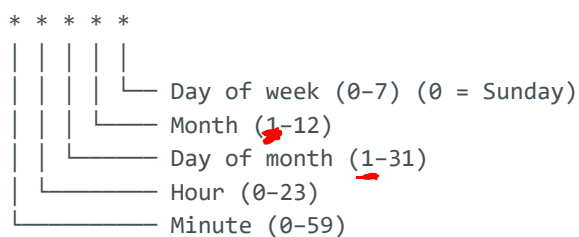
- A **scheduled pipeline** lets you run CI/CD jobs automatically at a specific time or interval (daily, weekly, every hour, etc.), similar to **cron jobs in Linux** .
- Example use cases:
 - Run nightly builds
 - Perform database backups
 - Run automated tests every Monday
 - Trigger deployments during off-peak hours

◆ How Scheduling Works in GitLab

1. Define jobs inside your **.gitlab-ci.yml** file.
2. Go to GitLab **UI** → **CI/CD** → **Schedules** to set when the pipeline should run.
3. GitLab uses **cron syntax** to specify schedules.

◆ Cron Syntax Refresher

Cron expressions follow this format:



✓ Examples:

- **0 0 * * *** → Run **every day at midnight**
- **0 3 * * 1** → Run **every Monday at 3 AM**
- ***/30 * * * *** → Run **every 30 minutes**
- **15 14 1 * *** → Run **at 2:15 PM on the 1st of every month**

◆ Example **.gitlab-ci.yml** with Scheduled Job

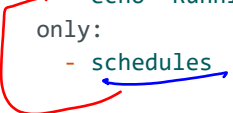
```

stages:
  - backup
  - test

backup_job:
  stage: backup
  script:
    - echo "Running database backup..."
    - pg_dump mydb > backup.sql
  only:
    - schedules #  runs only when triggered by schedule

test_job:
  stage: test
  script:
    - echo "Running scheduled tests..."
  only:
    - schedules

```



- Here, both jobs (`backup_job` and `test_job`) will only run when **pipeline is triggered by a schedule** , not when pushing code.

◆ Setting Up a Schedule in GitLab (GUI)

1. Go to your **project in GitLab** .
2. Navigate to **CI/CD** → **Schedules** .
3. Click **New Schedule** .
4. Enter:
 - **Description** (e.g., Nightly Backup)
 - **Cron expression** (e.g., `0 2 * * *` for daily at 2 AM)
 - **Timezone**
 - (Optional) **Custom variables** for the job
5. Save → GitLab will trigger pipelines automatically.

◆ Custom Environment Variables for Scheduled Jobs

You can add **variables** in schedules to control behavior:

Example:

- Add variable `BACKUP_TYPE=full` in the schedule.
- Access it in `.gitlab-ci.yml`:

```

backup_job:
  stage: backup
  script:
    - echo "Running $BACKUP_TYPE backup..."

```

◆ Sequential vs Parallel Schedules

- GitLab doesn't run schedules **in parallel** by default.
- If two schedules overlap:
 - A new pipeline is created even if the previous one is still running.
- You can control concurrency with **resource_group** or **when: delayed** if needed.

GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]

Test / Test / Schedules

Search or go to...

Merge requests 0

Manage >

Plan >

Code >

Build >

Pipelines

Jobs

Pipeline editor

Pipeline schedules

Artifacts

Secure >

Deploy >

Operate >

Monitor >

Analyze >

Settings >

Pipeline schedules

A scheduled pipeline starts automatically at regular intervals, like daily or weekly.

The pipeline:

- Runs for a specific branch or tag.
- Can have custom CI/CD variables.
- Runs with the same project permissions as the schedule owner.

Learn more in the [scheduled pipelines documentation](#).

Create a new pipeline schedule

49:38 / 1:14:33 • Schedule Job using Cron Gitlab >

https://gitlab.com/test6912220/Test/-/pipelines_schedules/new

GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]

Test / Test / Pipelines / Schedules

Search or go to...

Merge requests 0

Manage >

Plan >

Code >

Build >

Pipelines

Jobs

Pipeline editor

Pipeline schedules

Artifacts

Secure >

Deploy >

Operate >

Monitor >

Analyze >

Settings >

Schedule a new pipeline

Description

Custom Schedule

Interval Pattern

☐ Every day (at 2:01am)

☐ Every week (Saturday at 2:01am)

☐ Every month (Day 11 at 2:01am)

☒ Custom ?

12 ***

Set a custom interval with Cron syntax. [What is Cron syntax?](#)

Cron timezone

[UTC-7] Pacific Time (US & Canada)

Select target branch or tag

main

Variables

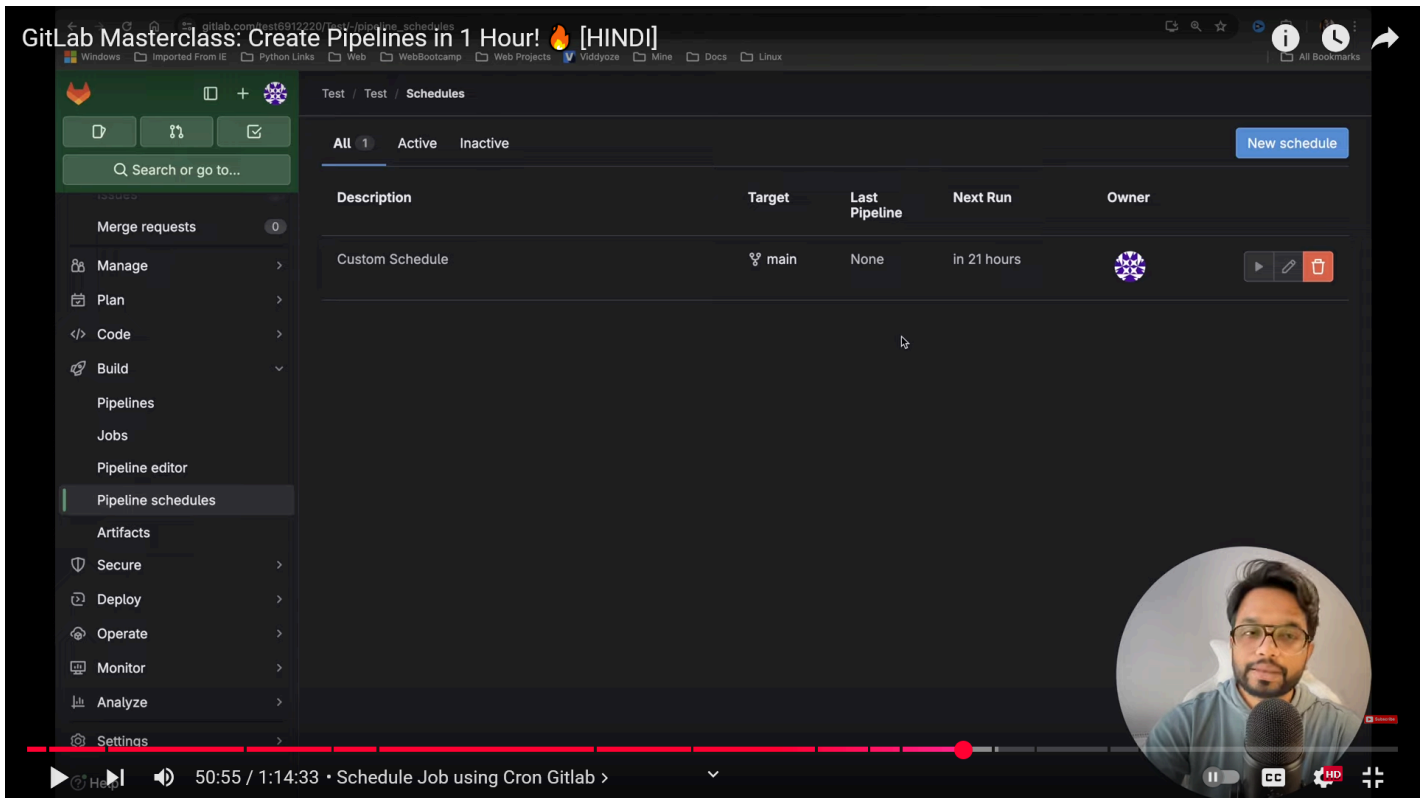
Variable Input variable key Input variable value

☒ Activated

Create pipeline schedule Cancel

50:49 / 1:14:33 • Schedule Job using Cron Gitlab >

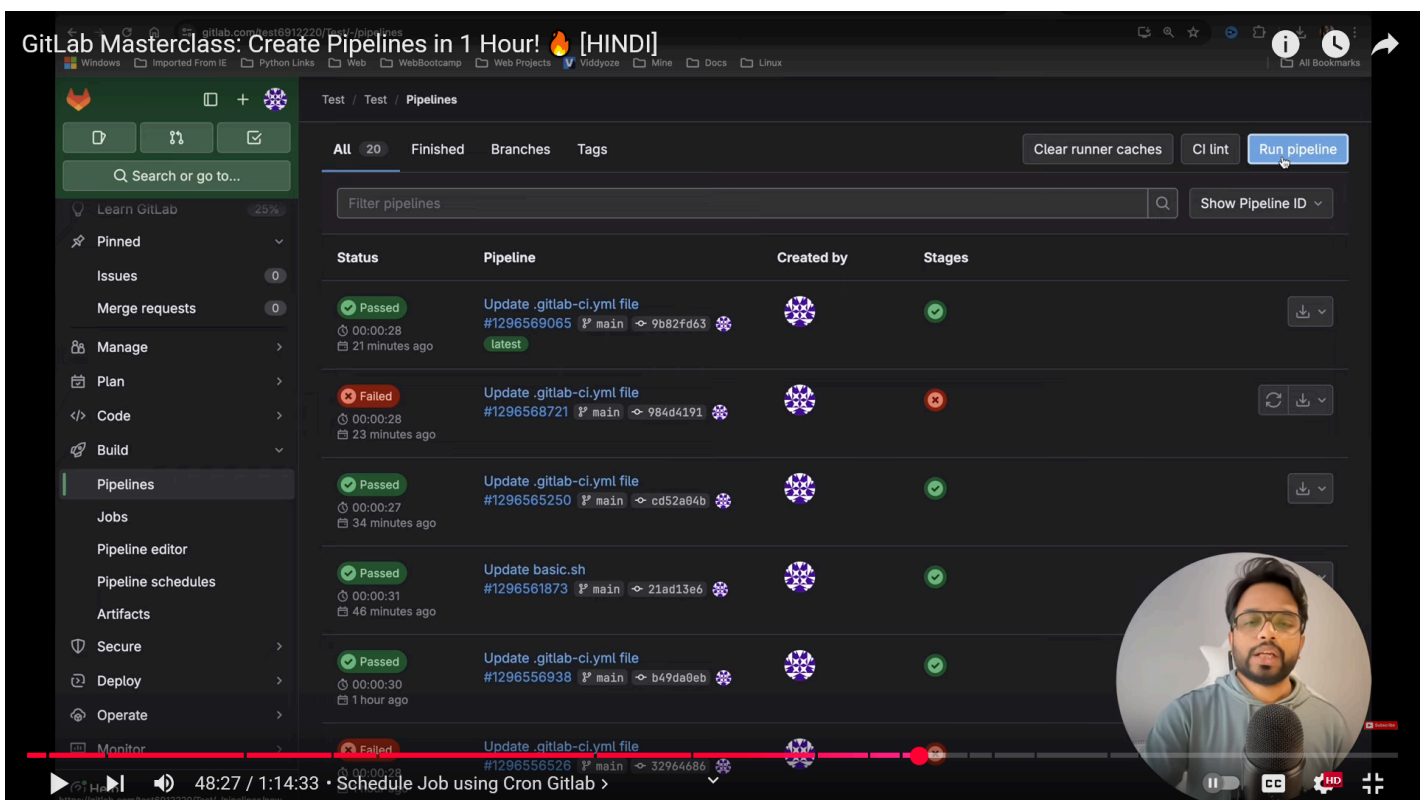
https://gitlab.com/test6912220/Test/-/pipelines_schedules/new



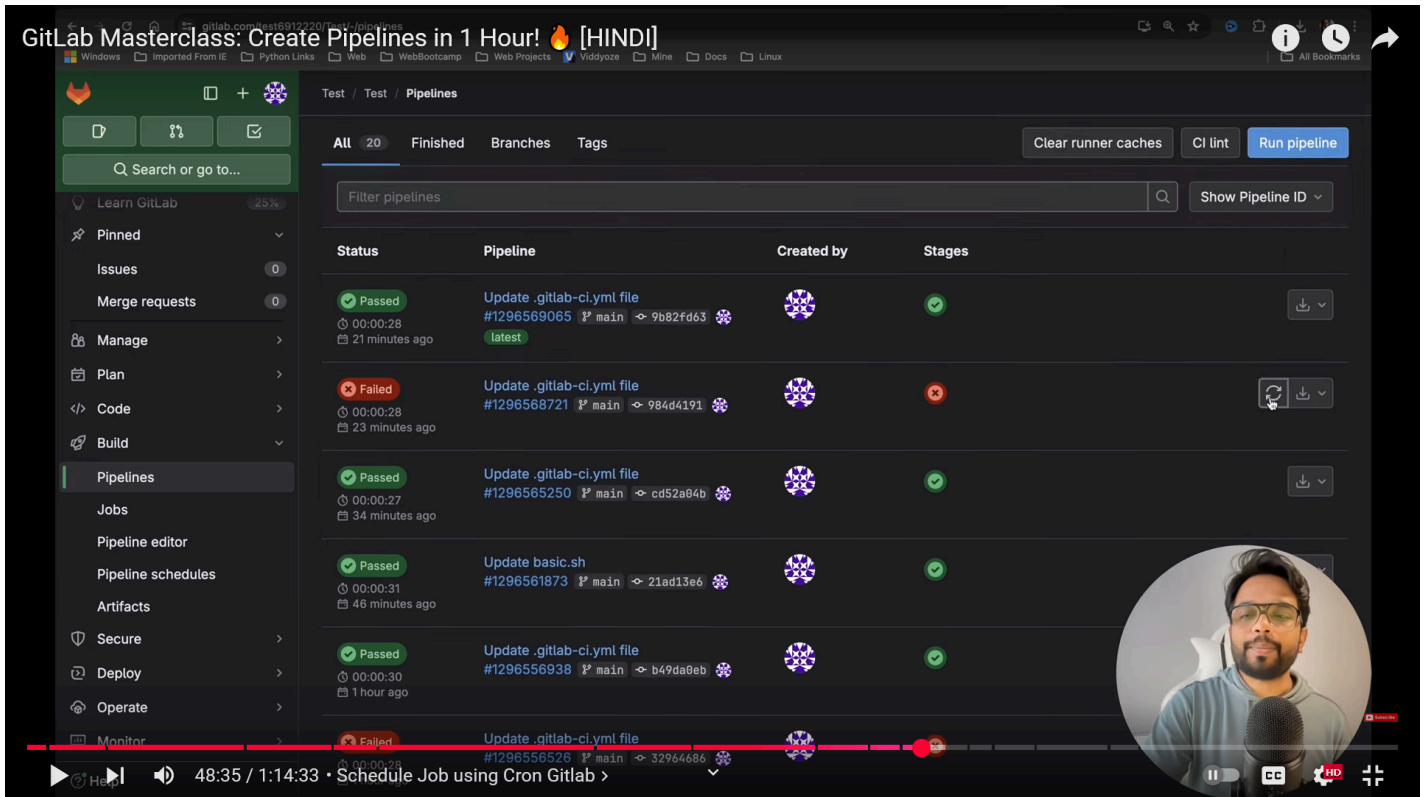
✓ In summary :

- Scheduled pipelines in GitLab are like cron jobs.
- Define jobs in `.gitlab-ci.yml`, but trigger them only with schedules (only: `[schedules]`).
- Use GitLab UI to configure cron expressions and variables.
- Great for nightly builds, tests, or backups.

----Running Pipelines Manually



Runing the failed job again --



----Manual Jobs and Manual Deployment

What is Manual Deployment in GitLab?

In GitLab CI/CD, a **manual deployment** means that a job won't run automatically when the pipeline executes. Instead, it waits for a user to manually trigger it from the GitLab UI (or API).

This is very useful for things like **production deployments**, where you don't want every commit to auto-deploy without approval.

How to Define Manual Jobs

In your `.gitlab-ci.yml`, you can mark a job as manual using:

```
deploy_production:
  stage: deploy
  script:
    - echo "Deploying to production..."
  when: manual
  environment:
    name: production
```

◆ Here:

- `when: manual` → tells GitLab this job must be started manually.

- It won't run automatically after previous jobs, but it **will wait in the pipeline** until someone clicks **Play** .

✅ Optional vs Required Manual Jobs

By default, manual jobs are **optional** .

- If skipped, the pipeline still passes.
- If you want to **require approval** before proceeding, you can add `allow_failure: false`.

```
deploy_production:
  stage: deploy
  script:
    - ./deploy.sh
  when: manual
  allow_failure: false
```

Now, the job must be triggered, otherwise the pipeline won't complete.

⚙️ Manual Jobs in Multi-Stage Pipelines

You might want automated tests → then manual deploy.

```
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script: echo "Building..."

test:
  stage: test
  script: echo "Running tests..."

deploy_staging:
  stage: deploy
  script: echo "Deploying to staging..."
  environment:
    name: staging
  when: manual

deploy_production:
  stage: deploy
  script: echo "Deploying to production..."
  environment:
    name: production
  when: manual
```

👉 Here, `deploy_staging` and `deploy_production` both require someone to **approve & trigger** manually.

Combining with Scheduled Pipelines

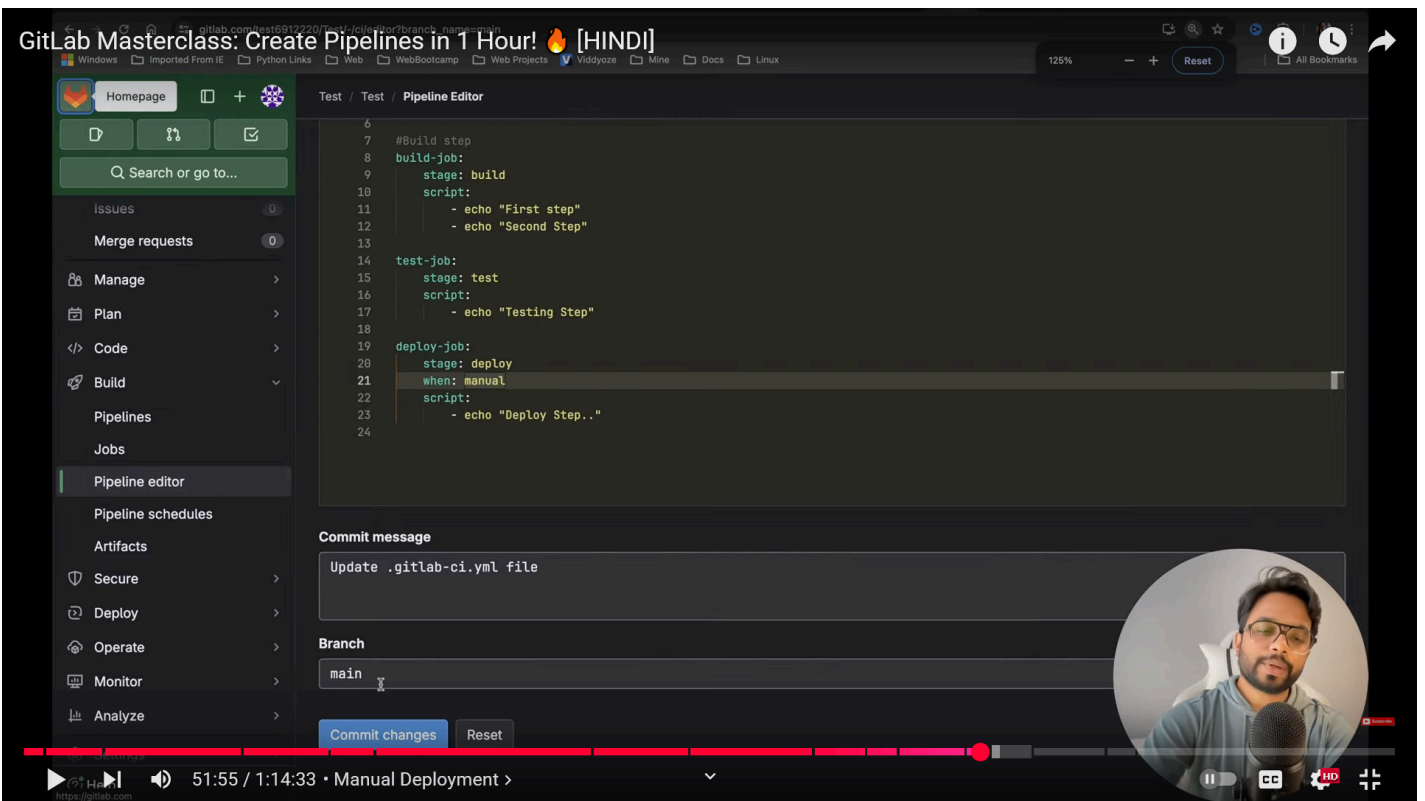
You can also schedule pipelines, where staging might run automatically but production needs manual trigger after review.

Permissions

Only users with enough permissions (e.g., **Developer** , **Maintainer** , or higher depending on project settings) can trigger manual jobs.

Triggering Manual Jobs

1. Go to your project in GitLab.
2. Navigate to **CI/CD** → **Pipelines** .
3. Find the running/completed pipeline.
4. For manual jobs, you'll see a **Play**  button .
5. Click it → job starts.

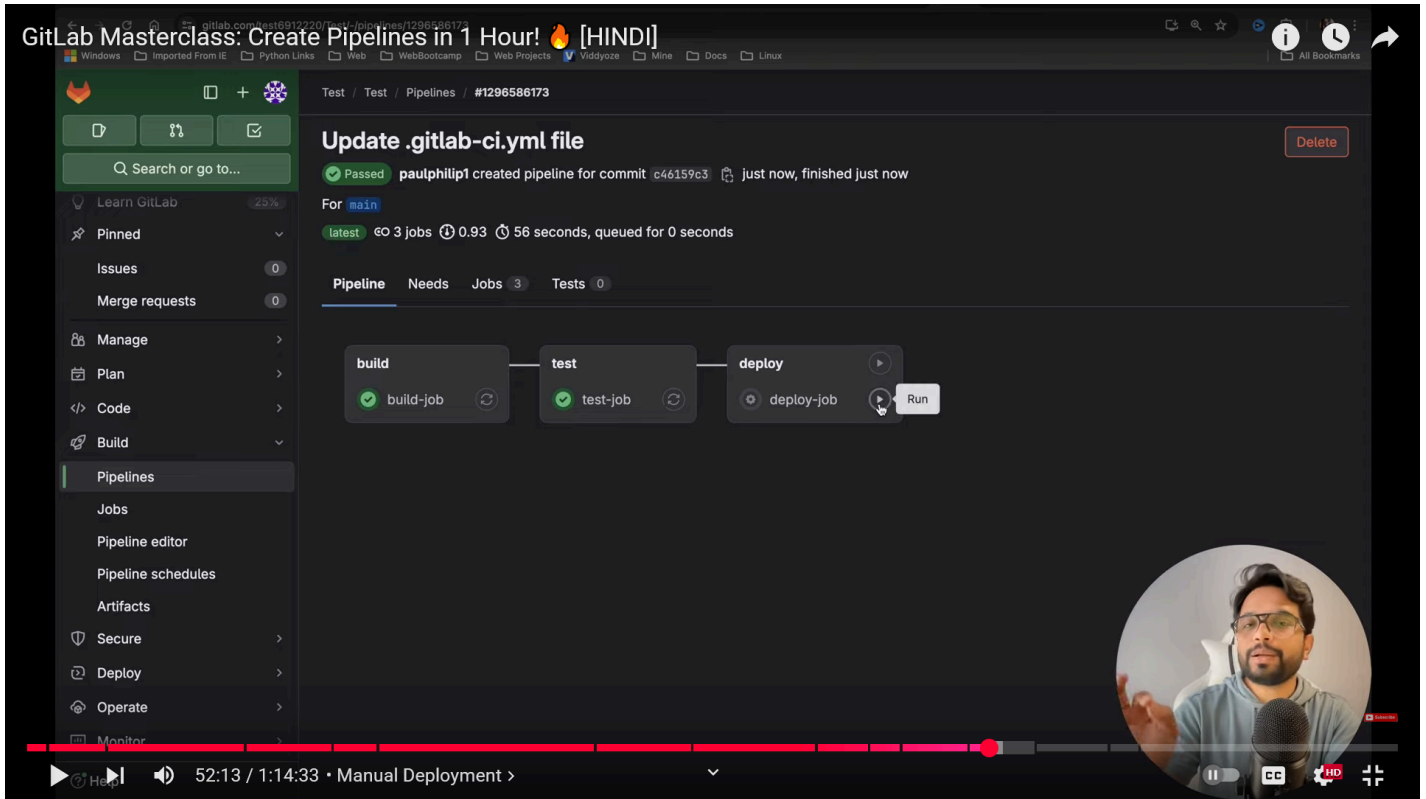


The screenshot displays the GitLab Pipeline Editor interface. The left sidebar contains navigation options: Homepage, Issues, Merge requests, Manage, Plan, Code, Build, Pipelines, Jobs, Pipeline editor (selected), Pipeline schedules, Artifacts, Secure, Deploy, Operate, Monitor, and Analyze. The main area shows the Pipeline Editor with a YAML configuration for a pipeline. The pipeline has three jobs: build, test, and deploy. The deploy job is configured with a manual trigger.

```
6
7 #Build step
8 build-job:
9   stage: build
10  script:
11    - echo "First step"
12    - echo "Second Step"
13
14 test-job:
15   stage: test
16   script:
17     - echo "Testing Step"
18
19 deploy-job:
20   stage: deploy
21   when: manual
22   script:
23     - echo "Deploy Step.."
24
```

Below the pipeline configuration, there is a section for the Commit message, which contains the text "Update .gitlab-ci.yml file". The Branch is set to "main". At the bottom, there are buttons for "Commit changes" and "Reset".

The video player interface at the bottom shows the video title "GitLab Masterclass: Create Pipelines in 1 Hour! 🔥 [HINDI]" and the current time "51:55 / 1:14:33". The video is titled "Manual Deployment".



----Predefined Variables in GitLab

◆ What are Predefined Variables in GitLab?

Predefined variables are **environment variables** that GitLab automatically provides during a pipeline/job run.

- They give you information about the **GitLab environment, repository, project, commit, user, runner, etc.**
- You can use them inside `.gitlab-ci.yml` without needing to define them yourself.

Think of them as GitLab's built-in context about your pipeline.

◆ Categories of Predefined Variables

GitLab organizes them into several groups:

1. Pipeline & Job Variables

- ✓ `CI_PIPELINE_ID` → Unique ID of the pipeline.
- ✓ `CI_JOB_ID` → Unique ID of the current job.
- ✓ `CI_JOB_STAGE` → Stage of the current job.
- `CI_JOB_NAME` → The name of the job.

* `CI_COMMIT_BRANCH`
* `CI_PIPELINE_SOURCE`

2. Repository & Git Variables

- `CI_COMMIT_REF_NAME` → Branch or tag name (e.g., `main`).
- `CI_COMMIT_SHA` → Full commit SHA (e.g., `a1b2c3d4`).
- `CI_COMMIT_SHORT_SHA` → Short commit SHA (8 chars).

- `CI_REPOSITORY_URL` → URL of the repo being built.

3. Project Variables

- `CI_PROJECT_NAME` → Project name.
- `CI_PROJECT_PATH` → Namespace/project path (e.g., `my-group/my-project`).
- `CI_PROJECT_DIR` → Full path where repository is cloned inside the runner.

4. User Variables

- `GITLAB_USER_ID` → ID of the user who triggered the pipeline.
- `GITLAB_USER_EMAIL` → Email of that user.
- `GITLAB_USER_LOGIN` → Username/login.

5. Runner Variables

- `CI_RUNNER_ID` → ID of the runner executing the job.
- `CI_RUNNER_EXECUTABLE_ARCH` → Architecture of runner (e.g., `linux/amd64`).

6. Environment & Deployment Variables

- `CI_ENVIRONMENT_NAME` → Name of environment (`staging`, `production`, etc.).
- `CI_ENVIRONMENT_URL` → URL linked to environment.

◆ Example Usage in `.gitlab-ci.yml`

```
stages:
  - build
  - deploy

build-job:
  stage: build
  script:
    - echo "Running on branch: $CI_COMMIT_REF_NAME"
    - echo "Commit SHA: $CI_COMMIT_SHORT_SHA"
    - echo "Pipeline ID: $CI_PIPELINE_ID"
    - echo "Project Name: $CI_PROJECT_NAME"

deploy-job:
  stage: deploy
  script:
    - echo "Deploying by user: $GITLAB_USER_EMAIL"
    - echo "Environment: $CI_ENVIRONMENT_NAME"
```

When this runs, GitLab replaces the variables with actual values.

For example:

```
Running on branch: main
Commit SHA: a1b2c3d4
Pipeline ID: 1245
Project Name: gym-ecommerce
Deploying by user: dev@example.com
Environment: production
```

◆ Why are Predefined Variables Useful?

- ✓ Debugging builds (know which commit/branch triggered it).
 - ✓ Deployments (e.g., different behavior for `main` vs `develop`).
 - ✓ Notifications/logging (show user email/username).
 - ✓ Accessing repo/project info without hardcoding.
-

----Importing Github project to GittLab

◆ Why Import GitHub → GitLab?

You may want to import when:

- Migrating from GitHub to GitLab (self-hosted or GitLab.com).
- Using GitLab CI/CD pipelines for automation.
- Consolidating projects in one place.

◆ Ways to Import a GitHub Project into GitLab

1. Using GitLab's GitHub Importer (Recommended)

GitLab has a built-in **GitHub integration** that allows you to directly import repositories.

Steps:

1. Go to your GitLab instance → **New Project** → **Import Project** → **GitHub** .
2. Authenticate with your **GitHub account** (OAuth or Personal Access Token).
 - If using a token, generate it from GitHub (**Settings** → **Developer Settings** → **Personal Access Token** with `repo` scope).
3. GitLab will list all your GitHub repositories.
4. Select the repository you want to import.
5. GitLab will create a project with:
 - Code (all commits, branches, tags).
 - Issues (if permissions allow).
 - Pull Requests → converted into Merge Requests.

2. Manual Import via Git Clone & Push

If you don't want to use GitHub's API:

Steps:

1. Clone your GitHub repo locally:

```
git clone --mirror https://github.com/username/repo.git
```

`--mirror` ensures all branches, tags, and refs are included.

2. Go to GitLab → **New Project** → Create a blank project.

3. Add GitLab as a remote and push everything:

```
cd repo.git
git remote set-url origin https://gitlab.com/username/repo.git
git push --mirror
```

✅ This copies the full Git history, branches, and tags.

❌ Issues, pull requests, and wiki won't migrate automatically.

3. Using GitHub → GitLab Migration Tools

- GitLab offers an **API-based migration tool** for enterprises.
- Some third-party tools (like `github-to-gitlab` scripts) exist for advanced migrations.

◆ Things to Consider

- **Private Repos:** Ensure GitHub token has `repo` scope.
- **CI/CD:** You'll need to manually configure `.gitlab-ci.yml` since GitHub Actions won't migrate.
- **Webhooks & Integrations:** Must be reconfigured manually.
- **Large Repos:** Use `git-lfs` if your project uses Large File Storage.

✅ Summary:

- **Quickest way** : GitLab's built-in GitHub importer.
- **Full control** : Clone + Push via `--mirror`.
- **Enterprise migration** : API-based or third-party tools.

----Running Job using Custom Image

When we say “**run a job using a custom image**” in GitLab CI/CD, the term *Docker image* comes up coz of the fact that GitLab **Runners** often use the **Docker executor** to run jobs.

Here's the breakdown:

1. Where Docker comes in

- GitLab Runner can run jobs in different ways (executors):

- **Shell** (runs commands directly on the host machine)
 - **Docker** (spins up a temporary Docker container per job)
 - **Kubernetes** , **VirtualBox** , etc.
- When you write this in `.gitlab-ci.yml`:

```
job1:
  image: node:18
  script:
    - node -v
```

It tells the GitLab Runner:

"If this Runner uses the Docker executor, please pull the `node:18` Docker image and run the job inside that container."

- That way, you don't need Node.js installed on the Runner host. The container provides a **pre-packaged environment** .

2. If you're not using Docker

- If your GitLab Runner is set up with the **Shell executor** , then the `image:` keyword will simply be ignored, because no Docker container is used.
- In that case, the job will run in the host machine's shell environment (whatever's already installed there).

3. Why use Docker images at all?

- **Consistency** : Jobs always run in the same environment (same Node.js, Python, etc.), no matter which Runner executes them.
- **Isolation** : Each job gets a fresh container, so no pollution from previous jobs.
- **Flexibility** : You can test different languages or versions just by changing the image.

Using Custom Docker Image

In GitLab CI/CD, you can **run jobs using a custom Docker image** by specifying the `image` keyword in your `.gitlab-ci.yml` file. This tells GitLab Runner to pull and use that Docker image as the execution environment for your job.

◆ How It Works

- By default, jobs run in the environment provided by the GitLab Runner.
- If you want specific tools, languages, or dependencies pre-installed, you can run jobs using **custom images** from:
 - Docker Hub (`node:18`, `python:3.11`, etc.)
 - GitLab Container Registry

- Any other private/public registry

◆ Example 1: Using a Public Docker Image

```
stages:
  - test

test-job:
  stage: test
  image: node:18    # Custom image from Docker Hub
  script:
    - node -v
    - npm install
    - npm test
```

➡ This job runs inside a **node:18** container with Node.js 18 pre-installed.

◆ Example 2: Using a Private Image from GitLab Container Registry

```
stages:
  - build

build-job:
  stage: build
  image: registry.gitlab.com/my-group/my-project/custom-image:latest
  script:
    - echo "Running inside custom image"
```

➡ Here, GitLab pulls your **own Docker image** from GitLab's built-in container registry.

◆ Example 3: Using Different Images for Different Jobs

```
stages:
  - lint
  - test
  - deploy

lint-job:
  stage: lint
  image: python:3.11
  script:
    - pip install flake8
    - flake8 .

test-job:
  stage: test
  image: node:18
  script:
    - npm install
    - npm test

deploy-job:
  stage: deploy
  image: alpine:latest
```

```
script:
  - echo "Deploying using Alpine"
```

➡ Each job runs in its **own container** with the required tools.

◆ Example 4: Using Services (DB + Custom Image)

You can combine a custom image with **services** (like databases):

```
stages:
  - test

test-job:
  stage: test
  image: node:18
  services:
    - name: postgres:15
  variables:
    POSTGRES_USER: user
    POSTGRES_PASSWORD: pass
    POSTGRES_DB: mydb
  script:
    - npm install
    - npm test
```

➡ Job runs inside **node:18**, and a **postgres:15** service container is linked for testing.

✅ Benefits of custom images :

- Consistent environments
- Faster jobs (dependencies pre-installed in the image)
- Avoids "works on my machine" issues

----Hosting Static Websites directly from your GitLab repository using **pages**

◆ What is GitLab Pages?

- **GitLab Pages** lets you **host static websites directly from your GitLab repository** .
- It's mainly used for:
 - Documentation sites (e.g., generated with Sphinx, MkDocs, Docusaurus).
 - Portfolio/personal sites.
 - Static site generators (Hugo, Jekyll, Gatsby, etc.).

GitLab Pages are deployed via **CI/CD pipelines** using a **special job named pages** inside **.gitlab-ci.yml**.

◆ How it works

1. You define a `pages` job in `.gitlab-ci.yml`.
2. That job collects the website files (HTML, CSS, JS, etc.) into a folder called `public/`.
3. GitLab automatically publishes the contents of `public/` as a website.

The `pages` job is special :

- Must be named `pages`.
- Must output artifacts with `public/` folder.
- GitLab will take `public/` and serve it as your website.

◆ Basic Example: Deploying a Static Website

```
stages:
  - build
  - deploy

pages:
  stage: deploy
  script:
    - mkdir .public
    - echo "<h1>Hello from GitLab Pages!</h1>" > .public/index.html
    - mv .public public # GitLab expects folder named "public"
  artifacts:
    paths:
      - public
  only:
    - main # deploy only on main branch
```

Explanation:

- `script` → generates an `index.html`.
- `artifacts.paths: public` → tells GitLab to take the `public/` folder as website files.
- The `only: main` ensures it only deploys when changes are pushed to the `main` branch.

◆ Where will the site be hosted?

- If your project is **public**, the URL looks like:

```
https://<username>.gitlab.io/<project-name>/
```

- If it's a **group project** :

```
https://<group-name>.gitlab.io/<project-name>/
```

✅ Key Rules for GitLab Pages Job

- Must be called **pages** .
- Must output artifacts containing a **public/** folder .
- GitLab serves only what's inside **public/**.

----All Stages, Jobs and reserved Keys in **.gitlab-ci.yml**

◆ 1. Built-in Jobs in **.gitlab-ci.yml**

There are a few **special job names** that GitLab recognizes automatically:

Job Name	Purpose
pages	Used to deploy content to GitLab Pages . Artifacts created by this job are published as a static site.
test	Not a reserved name, <u>but by convention</u> used for testing. (GitLab doesn't auto-recognize it, but pipelines often have test stage).
deploy	Similarly, a <u>convention</u> — but if placed in a deploy stage, GitLab UI highlights it as a deployment job.

👉 Only **pages** is a true “built-in job” with **special behavior** . Others are conventions but not mandatory.

◆ 2. Built-in Stages

By default, GitLab defines the following **pipeline stages** :

1. **build** – compile, package, or prepare artifacts.
2. **test** – run unit/integration tests.
3. **deploy** – deploy to an environment (staging/production).

✅ You can override or extend these by defining your own **stages:** list in **.gitlab-ci.yml**.

Example:

```
stages:
  - build
  - test
  - deploy
  - cleanup    # custom stage
```

◆ 3. Built-in Keys (keywords)

These are **reserved keywords** you can use in `.gitlab-ci.yml`:

Pipeline structure keys

- **stages**: → Defines the order of execution.
- **workflow**: → Controls when pipelines are created (rules, conditions).
- **include**: → Import other YAML files/templates.

Job-level keys

- **script**: → The commands to run inside the job.
- **stage**: → Assigns job to a stage.
- **only** / **except**: → Limit job execution by branch/tags/pipelines.
- **rules**: → Advanced conditional execution.
- **tags**: → Select specific runners.
 - **image**: → Use a custom Docker image.
- **services**: → Add service containers (e.g., DB).
- **variables**: → Define environment variables for the job.
- **cache**: → Reuse files between jobs.
 - **artifacts**: → Store job outputs.
- **dependencies**: → Specify which jobs' artifacts to use.
 - **before_script** / **after_script**: → Run before/after **script**.
- **needs**: → Run jobs in parallel with dependencies.
- **when**: → Control job execution (**on_success**, **on_failure**, **manual**, **delayed**).
- **environment**: → Define deployment environments (staging, production).
- **retry**: → Automatically retry failed jobs.
- **timeout**: → Job execution time limit.
 - **allow_failure**: → Allow job failure without failing pipeline.
- **extends**: → Reuse job configurations.

stages:
-
-

~~~~~:

stage:  
script:  
before-script:  
after-script:  
artifacts:  
when: manual  
allow-failure  
only: -schedules  
pages:  
image:

### Special keys

- **pages**: → GitLab Pages deployment job.
- **.hidden-job**: → Jobs prefixed with `.` are templates, not executed directly.

## ◆ 4. Example Full `.gitlab-ci.yml` Using Many Keys

```
stages:  
  - build  
  - test  
  - deploy  
  
variables:  
  NODE_ENV: "test"  
  
before_script:  
  - echo "Setting up..."
```

```

build-job:
  stage: build
  script:
    - echo "Building project..."
  artifacts:
    paths:
      - build/
    expire_in: 1 week

test-job:
  stage: test
  script:
    - echo "Running tests..."
  needs: ["build-job"]

deploy-job:
  stage: deploy
  script:
    - echo "Deploying app..."
  environment:
    name: production
  when: manual
  only:
    - main

pages:
  stage: deploy
  script:
    - echo "Publishing docs..."
  artifacts:
    paths:
      - public

```

## ✓ Summary:

- **Built-in Job** : `pages` (special meaning).
- **Built-in Stages** : `build`, `test`, `deploy` (default, can be overridden).
- **Built-in Keys** : `script`, `stage`, `artifacts`, `variables`, `rules`, etc.

# ----Keyword - `rules`

## ◆ What is `rules` in GitLab?

- The `rules` keyword is used inside a job in `.gitlab-ci.yml`.
- **It controls whether and when a job runs .**
- Rules are more powerful and flexible than `only/except` (which are older).
- You can define **conditions based on branch, pipeline type, file changes, variables, etc.**

## ◆ Structure

```

job_name:
  script:
    - echo "Run this job"
  rules:
    - if: '<condition>'
      when: <action>
      allow_failure: <true/false>

```

not necessary that  
if, when & allow-failure  
must be present, can be  
any one or more

## Actions for **when:**

- **on\_success** (default) → Runs if previous jobs succeed.
- **manual** → Requires a manual trigger in the GitLab UI.
- **always** → Runs regardless of status.
- **never** → Disables the job (useful for conditional skipping).
- **delayed** → Runs after a delay (with start\_in:).

• if  
• when  
• allow-failure  
• changed

## ◆ Examples

### 1. Run only on main branch

```

deploy:
  script: echo "Deploying..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

```

✓ The job runs only if the pipeline is on the **main** branch.

### 2. Run on merge requests

```

test:
  script: echo "Testing..."
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

```

✓ Runs only for merge request pipelines.

### 3. Run only if a file changes

```

lint:
  script: echo "Linting code..."
  rules:
    - changes:
      - "*.js"
      - "src/**/*"

```

✓ The job runs only when **.js** files or files in **src/** are modified.

## 4. Manual job (needs approval)

Eg ↴

```
deploy_prod:
  script: echo "Deploying to production..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
```

✅ Runs only on **main**, but requires a manual trigger in GitLab UI.

## 5. Multiple rules with priority

```
build:
  script: echo "Building..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always
    - if: '$CI_COMMIT_BRANCH == "dev"'
      when: manual
    - when: never
```

- Rule evaluation is **top-to-bottom** .
- First match wins.
- If no rule matches, job is skipped.

## 6. Delayed execution

```
notify:
  script: echo "Send notification after 30 min"
  rules:
    - when: delayed
      start_in: 30 minutes
```

✅ The job runs after 30 minutes automatically.

### ◆ Comparison: **rules:** vs **only/except**

- **only/except** is simple but limited.
- **rules:** can combine **branch**, **pipeline type**, **variables**, **file changes**, **manual/auto** behavior.

---

# ----Pipeline Structure key - **workflow**

### ◆ What is **workflow** in GitLab?

- **workflow:** defines **whether a pipeline should run at all** , and under what conditions.
- Think of it as a **global-level control** over pipelines (before jobs even start).



- It uses **rules** (similar to job-level **rules**) but applies to the entire pipeline.

📍 Location:

**workflow:** is defined at the **top level** of `.gitlab-ci.yml` (not inside a job).

## ◆ Syntax

```
workflow:
  rules:
    - if: <condition>
      when: <action>
```

- **if:** → condition (based on predefined variables, branch names, merge requests, etc.)
- **when:** → what to do (**always**, **never**)

## ◆ Examples

### 1: Run pipeline only on **main** branch

```
workflow:
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always
    - when: never
```

👉 Pipelines run only on **main**, all other branches won't trigger pipelines.

### 2: Run pipelines only for Merge Requests

```
workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      when: always
    - when: never
```

👉 Pipelines run only when an MR is created/updated, not for every push.

### 3: Skip pipelines for docs-only changes

```
workflow:
  rules:
    - changes:
        - "docs/**/*"
      when: never
    - when: always
```

👉 If the only changes are in `docs/`, pipeline won't run.

#### 4: Run pipeline for tags but not branches

```
workflow:
  rules:
    - if: '$CI_COMMIT_TAG'
      when: always
    - when: never
```

👉 Pipeline triggers only when you push a Git tag.

#### ◆ Difference: `workflow: rules` vs `job rules`

- `workflow: rules` → decides if the **pipeline should run at all** .
- `job: rules` → decides if a **particular job** should run **inside an already running pipeline** .

#### ✅ Summary:

- `workflow:` is like a **gatekeeper** at pipeline level.
- Uses `rules` with conditions (`if`, `changes`, `exists`).
- Prevents unnecessary pipelines (saving CI minutes).

---

## ----Keyword - `include`

---

In GitLab CI/CD, the `include` keyword lets you **reuse configurations** across multiple `.gitlab-ci.yml` files. Instead of repeating the same jobs or settings in every project, you can split them into separate files and include them.

#### ◆ Why use `include`?

- DRY principle ( **Don't Repeat Yourself** ).
- Share common CI/CD jobs between projects.
- Organize a large pipeline into smaller files.
- Use pre-defined templates from GitLab.

#### ◆ Types of `include`

You can include configs from different sources:

##### 1. Local file

- A `.yml` file inside the same repository.

```
include:
  - local: 'templates/common-jobs.yml'
```

## 2. File from another project

- Useful if you maintain a centralized CI/CD config in one project.

```
include:
  - project: 'group/ci-templates'
    file: '/pipelines/frontend.yml'
    ref: main # optional, specify branch or tag
```

## 3. Remote file (via URL)

- Import external `.yaml` file from the internet.

```
include:
  - remote: 'https://example.com/ci-templates/deploy.yml'
```

## 4. GitLab CI/CD templates

- GitLab provides ready-made templates (e.g., security scans, code quality).

```
include:
  - template: 'Security/SAST.gitlab-ci.yml'
```

### ◆ Example – Combining multiple includes

```
include:
  - local: 'jobs/test.yml'
  - project: 'devops/shared-ci'
    file: '/jobs/build.yml'
    ref: main
  - remote: 'https://example.com/templates/deploy.yml'
  - template: 'Security/SAST.gitlab-ci.yml'
```

### ◆ How it works

- GitLab fetches and **merges the included file(s)** with your `.gitlab-ci.yml`.
- If the same job or key is defined multiple times:
  - Local `.gitlab-ci.yml` has **priority** over included files.
  - Later includes override earlier ones.

### ✅ Use Case Example:

Let's say you want all your projects to run the same test job. You can keep it in a shared repo `ci-templates/test.yml` and include it everywhere, instead of copying.

---

## ----Keyword - tags

---

### ◆ What are Tags in GitLab CI/CD?

- In GitLab CI/CD, **tags** are used to tell **which runners** should pick up and execute a job.
- A runner can be registered with one or more tags.
- Jobs can also be assigned tags in `.gitlab-ci.yml`.
- If a job has tags, **only runners with all those tags** will be eligible to run the job.

👉 Think of tags as a **filter** between jobs and runners.

### ◆ Why Tags Are Needed?

- To control **where jobs run** .
- Example:
  - Some runners may have **Docker installed** .
  - Others may have **special tools (Java, Node, Python, etc.)** .
  - You don't want all jobs to run on all runners.

### ◆ Defining Tags for a Job

You specify tags under a job in `.gitlab-ci.yml` like this:

```
stages:
  - build
  - test

build-job:
  stage: build
  script:
    - echo "Building the app"
  tags:
    - docker
    - linux

test-job:
  stage: test
  script:
    - echo "Running tests"
  tags:
    - node
```

### ◆ Registering a Runner with Tags

When you register a runner (via `gitlab-runner register`), GitLab asks you for tags:

```
gitlab-runner register
Please enter the executor: shell, docker, etc: docker
Please enter the default Docker image: alpine:latest
Please enter the tags for this runner (comma-separated): docker, linux
```

Now this runner will only pick jobs with **tags: [docker, linux]**.

### ◆ Example Flow

1. You have 2 runners:

- Runner A (tags: **docker, linux**)
- Runner B (tags: **node**)

2. Jobs:

- **build-job** → has tags **docker, linux** → Runs on Runner A.
- **test-job** → has tag **node** → Runs on Runner B.

### ◆ Key Points

- If a job has **no tags**, then **any runner without tags** (shared runner usually) can pick it.
- If a job has tags, but **no runner matches them**, the job will stay **pending forever**.
- You can use tags to separate environments like **staging, production**, etc.

### ? Why can't we just use *one runner for everything* ?

In theory, you **can** have a single GitLab Runner installed on one powerful machine (or VM, or container) with **all dependencies/tools pre-installed**. That runner could technically pick up **any job** from any project.

But in practice, that creates **big issues** :

#### 1. Performance and Scalability

- One runner = single machine = limited CPU, memory, disk.
- If multiple jobs run in parallel, they will compete for resources.
- Large teams or CI/CD-heavy workflows would overwhelm a single runner.

👉 Multiple runners allow scaling — you can spin up many runners (even autoscaling with Kubernetes/Docker Machine/Cloud VMs) to handle workloads in parallel.

#### 2. Isolation

- Suppose Project A requires **Node.js 18**, Project B requires **Java 17**, and Project C requires **Python 3.12**.
- Installing everything on a single runner risks conflicts (e.g., different Node versions, dependency clashes, path conflicts).

- A job might “pollute” the environment for another job.

👉 With multiple runners (or using Docker executors with different images), each project/job gets a **clean, isolated environment** .

### 3. Security

- If all jobs run on a single runner, a malicious job from one project could access cached files, environment variables, or dependencies used by another project.
- This is especially risky if you use **shared runners** across multiple groups/users.

👉 Multiple runners = security isolation. You can dedicate runners to **trusted/internal projects** and separate them from **public/untrusted repos** .

### 4. Specialization

- Some jobs require **special hardware** (e.g., GPU for ML, macOS runner for iOS builds, Windows runner for .NET).
- One runner can't cover all specialized needs.
- Example:
  - A Linux runner builds backend services.
  - A Windows runner compiles .NET apps.
  - A macOS runner builds iOS apps.
  - A GPU runner trains ML models.

👉 Multiple runners allow **specialized execution environments** .

### 5. Maintenance

- If you put everything into one runner, maintaining it is a nightmare:
  - Every time a new project requires a new tool, you update the single runner.
  - Risk of breaking existing setups.
  - Hard to manage dependencies and versions.

👉 With multiple runners (and/or Docker images), you just pull a different image or assign a job to a different runner.

#### ◆ When does **one runner** make sense?

- Small teams, personal projects.
- Homogeneous stack (e.g., all Node.js projects with same version).
- Low job concurrency (not many parallel pipelines).
- No strict isolation/security needs.

#### ◆ When do you need **multiple runners** ?

- Large teams / many parallel pipelines.
- Different tech stacks (Java, Node, Python, Go, etc.).
- Projects with security isolation needs.
- Heavy CI/CD load (scaling across multiple machines/VMs).
- Special hardware needs (GPU, macOS, Windows).

### ✅ **Best practice :**

- Use **Docker executor runners** with pre-defined images (so every job gets its own clean environment).
- Use **multiple runners** for scaling, isolation, and specialization.
- For big setups, use **autoscaling runners** in the cloud (AWS, GCP, Kubernetes).

## 🔥 **More Details on Runners**

### 1. Types of GitLab Runners

When you install a GitLab Runner, you register it with an **executor** . The executor decides *how* the job will run. The common executors are:

- **Shell executor**
  - Runs jobs directly on the host machine (Linux, Windows, macOS).
  - No Docker needed.
  - Example: If your runner is installed on a Windows server, a job will run directly in Windows PowerShell or CMD. If it's installed on Linux, it will run in Bash.
  - ⚠️ Downside: jobs can interfere with each other since they all run on the same environment.
- **Docker executor**
  - Each job runs inside its own Docker container.
  - Isolated, clean environments for every run.
  - Requires Docker installed on the host runner.
- **VirtualBox, Kubernetes, SSH executors** (less common)
  - Can use VMs or Kubernetes pods for isolation.

### 2. So can I just use 1 runner with Linux or Windows?

Yes, you can:

- If you install GitLab Runner on a **Linux server** with the **shell executor** , then all jobs will run directly on Linux.
- If you install it on **Windows** with the shell executor, jobs will run in PowerShell or CMD.
- You don't *have* to use Docker — Docker is just popular because it gives clean and reproducible environments.

### ✅ Example use case :

- A **Windows runner** with shell executor → used for .NET apps.
- A **Linux runner** with shell executor → used for Node.js apps.
- A **Linux runner with Docker executor** → used when you want each job isolated in a container.

👉 So yes, you *can* run a GitLab runner purely on **Linux or Windows OS** without Docker.

## ----Keyword - services

### ◆ What are Services in GitLab CI/CD?

- A **service** is an **additional container** that runs alongside your **job container** in GitLab.
- It is mainly used when your job depends on **external tools** (like databases, caches, message queues) that you don't want to install in the main job container.
- Services are **temporary** – they live only for the duration of the job and are destroyed after it finishes.

### ◆ When do you need services?

Imagine:

- You want to test a backend app that needs a **PostgreSQL** or **MySQL** database.
- Or you want to test caching with **Redis** .
- Instead of installing PostgreSQL or Redis inside your main container, you tell GitLab to spin up a separate service container.

### ◆ Example: Using PostgreSQL as a Service

```
stages:
  - test

test_app:
  stage: test
  image: node:18 # Main job container where tests will run
  services:
    - postgres:13 # A PostgreSQL service container
  variables:
    POSTGRES_DB: testdb
    POSTGRES_USER: testuser
    POSTGRES_PASSWORD: testpass
  script:
    - npm install
    - npm run test
```



## Explanation:

- **image: node:18** → The main job runs inside a **Node.js container** .
- **services: - postgres:13** → A **Postgres database** container runs alongside it.
- **variables** → Environment variables are passed to the service container (so the app can connect to it).
- The Node app can now connect to **postgres:5432** (the hostname matches the service name).

## ◆ Multiple Services Example

```
integration_tests:
  stage: test
  image: python:3.10
  services:
    - postgres:13
    - redis:7
  variables:
    POSTGRES_DB: ci_db
    POSTGRES_USER: ci_user
    POSTGRES_PASSWORD: ci_pass
  script:
    - pip install -r requirements.txt
    - pytest
```

Here:

- The test job runs inside a **Python container** .
- Two services (**postgres** and **redis**) are running alongside.
- Tests can use both PostgreSQL and Redis.

## ◆ Key Points about Services

- **Services only work with Docker executors** (because GitLab uses Docker networking to link them).
- Each service container gets its own hostname (same as service name).
- Services are isolated per job (they don't persist across jobs unless you use **artifacts** or **cache** ).
- You can override the entrypoint/command if needed:

```
services:
  - name: mysql:8
    command: ["--default-authentication-plugin=mysql_native_password"]
```

## ✅ So in simple words:

**Services = Sidekick containers used for dependencies (like DBs, caches, queues) that your main job needs temporarily.**

# ----Keyword - cache

## ◆ What is cache in GitLab?

The **cache** keyword in `.gitlab-ci.yml` is used to **speed up pipelines** by storing files (like dependencies, libraries, build artifacts) so that future jobs or future pipeline runs can reuse them instead of downloading/rebuilding again.

Think of it as a **temporary storage** that GitLab saves between jobs and pipelines.

## ◆ Difference: cache vs artifacts

- **Cache** → For speeding up jobs by reusing dependencies. (Not meant for long-term storage; can be cleared anytime.)
- **Artifacts** → For storing build results/output (e.g., compiled binaries, test reports) to be **downloaded, browsed, or passed to other jobs**.

## ◆ Syntax of cache

```
job_name:
  script:
    - npm install
  cache:
    key: my-cache
    paths:
      - node_modules/
```

## Explanation:

- **key**: Uniquely identifies a cache. Jobs with the same **key** can reuse the cache.
- **paths**: Directories or files to store in cache.
- Cache is **shared across jobs** if the same key is used.

## ◆ Options for cache

### 1. key

Defines the cache identity.

Can be a string or a dynamic key (with predefined variables).

```
cache:
  key: "$CI_COMMIT_REF_SLUG"
```

```
paths:
  - node_modules/
```

- Here, each branch gets its own cache.

## 2. paths

List of directories/files to cache.

```
cache:
  paths:
    - vendor/
    - .m2/repository
```

## 3. policy

Controls **when** the cache is retrieved or uploaded.

- **pull-push** (default): Job **downloads** cache at start, and **uploads** updated cache at the end.
- **pull**: Job **only downloads** cache, doesn't update.
- **push**: Job **only uploads** cache, doesn't download.

```
cache:
  key: my-cache
  paths:
    - node_modules/
  policy: pull
```

## 4. untracked

Cache all **untracked files** in Git.

```
cache:
  untracked: true
```

## 5. when

Defines when caching happens:

- **on\_success** (default): Cache is saved if job succeeds.
- **always**: Cache is saved even if job fails.
- **on\_failure**: Cache is saved only when job fails.

```
cache:
  key: debug-cache
  paths:
```

```
- tmp/  
when: always
```

## ◆ Example: Caching Dependencies

### Node.js Example

```
cache:  
  key: npm-cache  
  paths:  
    - node_modules/  
  
install_dependencies:  
  script:  
    - npm install
```

### Python Example

```
cache:  
  key: pip-cache  
  paths:  
    - .venv/  
    - .cache/pip
```

### Maven Example

```
cache:  
  key: maven-cache  
  paths:  
    - .m2/repository
```

## ◆ Summary

- **Cache = speed optimization.**
- Stored on GitLab runners between jobs and pipelines.
- **Main keys :**
- **key:** Identify cache (can be dynamic).
- **paths:** What to cache.
- **policy:** Control push/pull.
- **when:** When to save cache.
- **untracked:** Cache git-untracked files.

---

# ----Keyword - dependencies

---

- **Purpose** : Controls which jobs' **artifacts** are downloaded for a given job.
- By default, a job **downloads all artifacts** from jobs in previous stages.
- With **dependencies**, you can **narrow it down** to specific jobs → this saves time, bandwidth, and storage.

#### ✓ Example:

```
stages:
  - build
  - test

build-job:
  stage: build
  script:
    - echo "Building..."
    - echo "artifact" > build.txt
  artifacts:
    paths:
      - build.txt

test-job:
  stage: test
  script:
    - cat build.txt
  dependencies:
    - build-job # Only download artifacts from build-job
```

👉 Without **dependencies**, **test-job** would try to fetch artifacts from **all jobs in previous stages** .

👉 With **dependencies**, we restrict it to just **build-job** .

---

## ----Keyword - **needs**

- **Purpose** : Defines **job dependencies for execution order** (not artifacts).
- Normally, jobs run **stage by stage** → all jobs in **build** finish before **test** starts.
- With **needs**, you can run a job **as soon as its required jobs are done** , even if earlier stage jobs are still running → enables **parallel execution** .
- Can also **fetch artifacts** from the **needs** job.

#### ✓ Example:

```
stages:
  - build
  - test
  - deploy

build-job:
  stage: build
  script:
    - echo "Build done"
    - echo "artifact" > build.txt
```

```

artifacts:
  paths:
    - build.txt

test-job:
  stage: test
  script:
    - cat build.txt
  needs:
    - job: build-job    # This allows test-job to run immediately after build-job

deploy-job:
  stage: deploy
  script:
    - echo "Deploying..."
  needs:
    - job: test-job

```

👉 Here, **test-job** does **not wait** for other **build** stage jobs to finish → it only waits for **build-job**.

👉 This makes the pipeline **faster** (DAG execution model).

## ----Keyword -- **retry** and **timeout**

In **GitLab CI/CD** , both **retry** and **timeout** are job-level keywords that control **job execution behavior** when things go wrong or take too long.

### ◆ **retry** keyword

The **retry** keyword defines how many times GitLab should **re-run a job if it fails** .

✅ When retry happens?

- Job **fails** because of:
  - Runner system failure
  - Job script error
  - Network or API failure
  - Exit code ≠ 0

❌ When retry does NOT happen?

- Job is **manually canceled** .
- Job fails because of **timeout** .
- Job is skipped due to rules/only/except.

**Syntax:**

```
job_name:
  script: npm install
  retry: 2
```

👉 Here, GitLab will try the job **up to 2 extra times** if it fails (so it could run max 3 times: original + 2 retries).

#### Advanced Usage (with conditions):

```
job_name:
  script: npm install
  retry:
    max: 3
    when:
      - runner_system_failure
      - stuck_or_timeout_failure
```

- **max**: maximum retry attempts.
- **when**: conditions that trigger retries (examples: **always**, **script\_failure**, **api\_failure**, **runner\_system\_failure**).

#### 💠 **timeout** keyword

The **timeout** keyword defines how long a job is allowed to **run before GitLab kills it** .

#### Syntax:

```
job_name:
  script: run-heavy-task.sh
  timeout: 30m
```

👉 This job will be killed if it runs longer than **30 minutes** .

You can use:

- **s** → seconds
- **m** → minutes
- **h** → hours

Default timeout = **1 hour** (if not set at runner/project level).

#### 🔑 Key Difference

- **retry**: Controls **how many times to try again** after failure.
- **timeout**: Controls **how long a job is allowed to run** before failure.

## ✅ Example combining both:

```
build:
  stage: build
  script:
    - make build
  retry: 2          # Retry job 2 times if it fails
  timeout: 45m      # Kill job if it runs more than 45 minutes
```

---

# ----Keyword - environment

---

The **environment** keyword in `.gitlab-ci.yml` is used to define a **deployment environment** (e.g., *staging*, *production*, *review apps*).

It lets GitLab keep track of **where your app is deployed** and manage deployment environments through the **Environments Dashboard**.

## ✅ Syntax:

```
deploy_job:
  stage: deploy
  script:
    - ./deploy.sh
  environment:
    name: production
    url: https://myapp.com
```

## 🔑 Key Options:

- **name** → The environment name (e.g., *staging*, *production*, *review/\**).
- **url** → The deployed app's URL (displayed in GitLab UI).
- **on\_stop** → Job to stop the environment (useful for review apps).
- **action** :
- **start** → Default, when deploying.
- **stop** → Used when shutting down an environment.

## 🚀 Example with review apps:

```
deploy_review:
  stage: deploy
  script:
    - ./deploy_review.sh
  environment:
    name: review/$CI_COMMIT_REF_NAME
    url: https://$CI_COMMIT_REF_NAME.review.myapp.com
    on_stop: stop_review
```



```
stop_review:
  stage: cleanup
  script:
    - ./stop_review.sh
  environment:
    name: review/$CI_COMMIT_REF_NAME
    action: stop
  when: manual
```

◆ Here, GitLab creates a **dynamic environment** for each branch (`review/branch-name`) and provides a stop button.

---

## ----Keyword - `extends`

---

The `extends` keyword is used to **reuse configuration** between multiple jobs.

It helps reduce duplication in `.gitlab-ci.yml`.

### ✓ Syntax:

```
.base_job:
  script:
    - echo "This is a base job"
  tags:
    - docker
  retry: 2

test_job:
  extends: .base_job
  script:
    - echo "This is test job"
```

### 🔑 Key Points:

- Jobs can **inherit** from one or more base jobs.
- If a job **overrides** a key (like `script`), it replaces the inherited one.
- Supports **merging** :
  - Scalars (like `tags`) → replaced.
  - Arrays (like `script`) → concatenated.

#### Scalars and Arrays

When a job **extends** another job/template:

- **Scalars (single-value keys like `image`, `stage`, `timeout`, `environment`, etc.)** → **replaced**

The child job overrides the parent value.

- **Arrays (multi-value keys like `script`, `before_script`, `after_script`)** → **concatenated**

Instead of replacing, GitLab **merges them** by concatenation (parent's values first, then child's).

Example:

```
.default-job:
  image: node:18
  script:
    - echo "from parent"
  tags:
    - docker

child-job:
  extends: .default-job
  script:
    - echo "from child"
  tags:
    - linux
```

**What happens here:**

- **image** → scalar → replaced (final = `node:18`, but if child gave a new one, it would overwrite).
- **script** → array → concatenated → final =

```
script:
  - echo "from parent"
  - echo "from child"
```

- **tags** → array BUT special case → replaced (not concatenated).

✅ So **most scalars get replaced, arrays get merged**, except for some special cases (`tags`, `artifacts`, `cache` → replaced).

 **Example:**

```
.default_template:
  image: node:16
  before_script:
    - npm install
  cache:
    paths:
```

```
    - node_modules/

test_job:
  extends: .default_template
  script:
    - npm test

lint_job:
  extends: .default_template
  script:
    - npm run lint
```

◆ Both `test_job` and `lint_job` reuse the same base configuration (image, setup, cache) but run different scripts.

---

## ----Registering a GitLab Runner on an **EC2** instance (Or any server)

---

### ◆ 1. Prerequisites

- An **AWS EC2 instance** (Ubuntu is easiest).
- GitLab project or group where you want the runner.
- **Runner registration token** (you get this from GitLab UI).

- **Project-level runner** :

Settings → CI/CD → Runners → Expand → Register a runner

- **Group/instance-level runner** : Similar place depending on scope.

### ◆ 2. Install GitLab Runner on EC2

SSH into your EC2 instance:

```
# Update
sudo apt-get update -y

# Install dependencies
sudo apt-get install -y curl apt-transport-https

# Add GitLab official repo
curl -L https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh |
sudo bash

# Install GitLab Runner
sudo apt-get install -y gitlab-runner
```

Check version:

```
gitlab-runner --version
```

### ◆ 3. Register the Runner

Run:

```
sudo gitlab-runner register
```

It will ask you for details:

1. **GitLab URL** → enter your GitLab instance URL  
(for GitLab.com → <https://gitlab.com/>)
2. **Registration token** → paste the one you copied.
3. **Description** → any name for your runner, e.g., [ec2-runner](#).
4. **Tags** → (optional) add tags like [aws](#), [ec2](#), [docker](#) if you want to limit jobs to specific runners.
5. **Executor** → choose how jobs should run:
  - [shell](#) → simplest (runs directly on EC2)
  - [docker](#) → most common (isolated jobs inside Docker containers)
  - [docker+machine](#) → autoscaling runners (advanced)

### ◆ 4. Configure the Runner

Configuration is saved in:

```
/etc/gitlab-runner/config.toml
```

Example for a **docker executor** :

```
[[runners]]
  name = "ec2-runner"
  url = "https://gitlab.com/"
  token = "YOUR_REGISTERED_TOKEN"
  executor = "docker"
[runners.docker]
  tls_verify = false
  image = "alpine:latest"
  privileged = true
  disable_entrypoint_overwrite = false
  oom_kill_disable = false
```

```
disable_cache = false
volumes = ["/cache"]
```

## ◆ 5. Start and Enable Runner

```
sudo gitlab-runner start
sudo systemctl enable gitlab-runner
```

Check if it's running:

```
gitlab-runner status
```

## ◆ 6. Verify in GitLab

- Go back to **Project** → **Settings** → **CI/CD** → **Runners**
- You should now see your EC2 runner listed as "active".

---

# -----MongoDb Atlas-----

## ----To Learn - Topics

### ◆ Beginner (Getting Started with MongoDB Atlas)

- **Introduction to MongoDB Atlas**
  - What is Atlas and why use it over self-hosted MongoDB?
  - Free tier vs paid tiers (cluster sizes, limitations).
- **Cluster Basics**
  - Creating a free-tier cluster (M0 Sandbox).
  - Cluster regions (choosing AWS, GCP, or Azure).
  - Understanding cluster tiers (M0, M10, dedicated clusters).
- **Database Connection**
  - Connecting Atlas with Compass, VSCode, and Node.js apps.
  - Connection string (SRV URI) structure.
  - IP Whitelisting basics.
- **Atlas Dashboard**

- Cluster overview, monitoring metrics.
- Basic UI navigation.

## ◆ Intermediate (For Real-World Projects)

- **Cluster Management**
  - Scaling up/down clusters.
  - Cluster sharding and replication basics.
  - Performance considerations (IOPS, storage, compute).
- **Database Security**
  - Database users & roles (read/write, custom roles).
  - IP Access List (CIDR ranges, VPC peering).
  - TLS/SSL encryption basics.
- **Backup & Restore**
  - Snapshots and point-in-time recovery.
  - Automated backups.
- **Monitoring & Performance**
  - Real-time performance metrics.
  - Profiler integration (slow queries, query optimization).
- **Atlas Search**
  - Creating search indexes.
  - Text search, autocomplete, fuzzy matching.
- **Integrations**
  - MongoDB Atlas with GitHub Actions / GitLab CI/CD.
  - Exporting/importing data with MongoDB Tools.

## ◆ Advanced (For Production & Scaling)

- **Global Deployment**
  - Multi-region clusters.
  - Read/write distribution.
  - Data locality compliance (GDPR, Indian data laws).
- **Advanced Security**
  - VPC Peering.
  - Private Endpoints (AWS PrivateLink, GCP Private Service Connect, etc.).
  - Field-level encryption (client-side).
  - Key Management System (KMS) integration (AWS KMS, Azure Key Vault).
- **Scalability & Performance**
  - Sharding (choosing shard keys).
  - Advanced indexing strategies (wildcard, compound, text, hashed indexes).
  - Query performance optimization (covered queries, explain plans).
- **Data Tools**
  - Atlas Data API (serverless CRUD without SDKs).
  - Realm Triggers (event-driven functions).

- Realm Sync (for mobile apps).
  - **Advanced Integrations**
    - Atlas + AWS Lambda / GCP Functions / Azure Functions.
    - Atlas with BI tools (Tableau, Power BI).
    - Atlas + Kafka (Change Streams → Kafka).
  - **Disaster Recovery**
    - Multi-cloud redundancy.
    - Failover strategies.
- 

# ----Introduction

---

## 1. What is MongoDB Atlas?

- MongoDB Atlas is a **fully managed cloud database service** provided by MongoDB Inc.
- Instead of you installing MongoDB locally or managing a server, Atlas runs it on cloud providers ( **AWS, GCP, Azure** ) and handles:
  - Deployment
  - Scaling
  - Backups
  - Security
  - Monitoring
  - Global distribution

In short: *Atlas* = *MongoDB without server headaches*.

## 2. Core Features

Here's what makes Atlas powerful:

### Deployment & Clusters

- You create **clusters** (groups of MongoDB servers).
- Cluster Types:
  - **M0 (Free Tier)** – Shared cluster, great for dev/testing.
  - **Dedicated Clusters (M10 and above)** – For production.
  - **Serverless Instances** – Pay-per-use, good for unpredictable workloads.
- You choose:
  - Cloud provider (AWS, GCP, Azure)
  - Region (closest to your users)
  - Instance size (RAM, storage, vCPUs)

### Global Distribution

- **Multi-region clusters** → replicate data across continents.
- Enables:
  - Low-latency reads/writes
  - Failover (automatic if one region fails)
  - Data locality (keep data near users for compliance, e.g., GDPR)

## ✅ Security

- **Authentication :**
- Username/Password
- SCRAM, X.509, LDAP, SAML
- **Network Access :**
- IP Whitelisting
- VPC Peering / Private Endpoints
- **Encryption :**
- In-transit (TLS/SSL)
- At-rest (disk encryption)
- Customer Key Management (KMS with AWS/GCP/Azure)

## ✅ Monitoring & Automation

- Atlas monitors CPU, memory, queries, slow operations.
- Automatic:
  - Backups & Point-in-Time Recovery
  - Scaling (RAM, storage, nodes)
  - Replica set elections during failures

## ✅ Database Services

- **Data API** – Access your cluster over HTTPS without drivers.
- **Atlas Charts** – Built-in visualization dashboard.
- **Atlas Search** – Full-text search using Apache Lucene (no separate Elasticsearch needed).
- **Atlas Triggers** – Run serverless functions on DB changes (like `changeStream` with cloud functions).
- **Atlas Functions (Realm)** – Serverless backend for logic + GraphQL API.
- **Atlas Device Sync** – Sync data with mobile devices.

## 3. Cluster Architecture

- **Replica Sets** (minimum 3 nodes):
  - Primary → handles writes
  - Secondary → handles reads, failover
  - Arbiter (optional) → voting in elections
- **Sharding :**
- For large-scale data



- Distributes collections across shards based on shard key
- Router (**mongos**) handles query routing

## 4. Working with Atlas

### ◆ Steps to Use Atlas with MERN

#### 1. Create Cluster

- Sign in to [MongoDB Atlas](#).
- Choose cloud provider + region.
- Select free/shared/dedicated cluster.

#### 2. Set Security

- Add **IP whitelist** (your dev machine/server).
- Create **database user** with username/password.

#### 3. Connect to Cluster

- Get connection string (URI):

```
mongodb+srv://username:password@cluster0.abcd.mongodb.net/myDatabase?
retryWrites=true&w=majority
```

- Use it in your MERN backend (Node.js with Mongoose):

```
import mongoose from "mongoose";

mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.error(err));
```

#### 4. Deploy Application

- Host backend/frontend (e.g., EC2, Vercel, Netlify).
- Connect to Atlas securely.

## 5. Scaling in Atlas

- **Vertical Scaling** → Increase cluster tier (RAM, CPU, IOPS).
- **Horizontal Scaling (Sharding)** → Add more nodes for massive data.
- **Auto-scaling** → Storage and compute adjust automatically.

## 6. Advanced Features

- **Atlas Data Federation** → Query data across S3, Atlas, and other sources with a single query.
- **Atlas Vector Search** → Store and query embeddings (AI/ML workloads).

- **Atlas Online Archive** → Move old data to cheaper storage (S3) while keeping queryable.
- **Performance Advisor** → Suggests indexes automatically.
- **Backup & Restore** → Continuous backups with point-in-time restore.

## 7. Use Cases

- Web apps (MERN, MEAN, Next.js)
- Mobile apps (sync with Realm/Device Sync)
- IoT (real-time streaming data)
- Analytics (Atlas Charts, BI Connector for Tableau/PowerBI)
- AI/ML (Vector Search, embeddings storage)

## 8. Benefits over Self-Hosting MongoDB

| Self-Hosted MongoDB                  | MongoDB Atlas                |
|--------------------------------------|------------------------------|
| You manage servers, updates, backups | Fully managed service        |
| Complex sharding & scaling           | One-click sharding/scaling   |
| Manual security setup                | Built-in security defaults   |
| Monitoring needs third-party tools   | Built-in monitoring & alerts |
| Limited global presence              | Multi-region clusters        |

## 9. Pricing

- Free Tier (M0) – 512MB storage.
- Dedicated Clusters – pay per hour (varies by region & tier).
- Serverless – pay only for reads/writes/storage.

## 10. Where to Learn Atlas Hands-On

- MongoDB University (free courses).
- Atlas Docs: <https://www.mongodb.com/docs/atlas/>.
- Try free cluster → connect it to your MERN project.

✅ In short: **MongoDB Atlas = MongoDB + Cloud Native + Automation + Global Scale.**



**IMP--You just focus on building apps (like FitLab MERN project ) instead of worrying about managing databases.**

MongoDB Atlas itself is **not the server** . It's a **software-as-a-service (SaaS)** platform that sits on top of cloud infrastructure like **AWS, Azure, or GCP**

Here's the breakdown:

## ◆ How It Works

- **Cloud Provider (AWS/Azure/GCP)**

- Provides the **physical servers, storage, and networking** .
- Example: If you pick AWS + Mumbai region → Atlas provisions EC2 instances + EBS volumes behind the scenes.

- **MongoDB Atlas Layer**

- Installs & configures MongoDB on those servers.
- Adds management tools: scaling, monitoring, security, backups.
- Provides a dashboard + APIs so you never have to manually touch those AWS/Azure/GCP servers.

## ◆ Example

Suppose you create a cluster in Atlas:

1. You select **AWS** as the cloud provider.
2. Atlas spins up **EC2 instances** in AWS, attaches **EBS storage** , and configures networking.
3. On top of that, it deploys **MongoDB replica sets / sharded clusters** .
4. You just see a connection string like:

```
mongodb+srv://user:pass@cluster0.abcde.mongodb.net/myDB
```

→ You never log in to the EC2 instances directly. Atlas abstracts that away.

## ◆ Analogy

Think of it like this:

- **AWS/Azure/GCP** = the land and building materials.
- **MongoDB Atlas** = the builder + property manager who constructs the house, maintains it, secures it, and gives you the keys to live inside.
- You = the resident who just uses the house (database) without worrying about plumbing, wiring, or repairs.

👉 So yes, **the real servers belong to AWS/Azure/GCP** , but **Atlas manages them for you** with an extra layer of automation and features.

---

---

---

---

# -----Hosting using EC2, NGINX, S3, Cloudfront, Route 53, GitLab-----

---

## Architecture (what goes where)

- **Frontend (React + framer-motion + tailwind)** → **S3** bucket (private) → **CloudFront** CDN (public) → **Route 53** DNS ([fitlab.com](https://fitlab.com), [www.fitlab.com](https://www.fitlab.com)).
- **Backend (Express + JWT + Socket.IO + multer + Cloudinary)** → **EC2 (Ubuntu)** running Node/PM2 behind **NGINX** ([api.fitlab.com](https://api.fitlab.com)) → **Route 53** DNS.

Tip: Use an **Elastic IP** for EC2 so DNS doesn't break on reboots.

- **Database** → **MongoDB Atlas** (don't host Mongo on EC2).
- **Realtime** → Socket.IO proxied by NGINX; **WebRTC** needs STUN/TURN (see "WebRTC nuances" below).

## Prerequisites (one-time)

- **Domain** in Route 53 (or imported).
- **ACM cert (us-east-1)** for [fitlab.com](https://fitlab.com), [www.fitlab.com](https://www.fitlab.com) (CloudFront).
- **Let's Encrypt** cert on EC2 for [api.fitlab.com](https://api.fitlab.com) (NGINX).
- **IAM**
  - CI user with minimal permissions: `s3:PutObject`, `s3:ListBucket`, `cloudfront:CreateInvalidation`.
  - EC2 role (instance profile) with CloudWatch logs (optional).
- **Secrets** ready (GitLab → Settings → CI/CD → Variables):
  - `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_DEFAULT_REGION`
  - `S3_BUCKET=fitlab-frontend-prod`
  - `CLOUDFRONT_DISTRIBUTION_ID=E123...`
  - `EC2_HOST=api.fitlab.com` (or Elastic IP), `EC2_USER=ubuntu`, `EC2_SSH_PRIVATE_KEY` (masked)
  - App envs: `MONGODB_URI`, `JWT_SECRET`, `CLOUDINARY_URL`, etc.

## EC2 (Ubuntu) — backend host

1. **Launch** t3.small (example) + **Security Group** : `22` (your IP), `80`, `443`.

2. **SSH in & base setup**

```
sudo apt update && sudo apt -y upgrade
sudo apt -y install nginx ufw git build-essential
```

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt -y install nodejs
sudo npm i -g pm2@latest
sudo ufw allow OpenSSH && sudo ufw allow 'Nginx Full' && sudo ufw --force enable
```

### 3. Clone & run your API

```
mkdir -p /var/www/fitlab-api && cd /var/www/fitlab-api
git clone <your-repo> .
npm ci
pm2 start ecosystem.config.js --env production
pm2 save && pm2 startup systemd -u $USER --hp $HOME
```

#### Example `ecosystem.config.js`:

```
module.exports = {
  apps: [{
    name: "fitlab-api",
    script: "dist/server.js",
    instances: "max",
    exec_mode: "cluster",
    env_production: {
      NODE_ENV: "production",
      PORT: 3000,
      MONGODB_URI: process.env.MONGODB_URI,
      JWT_SECRET: process.env.JWT_SECRET,
      CLOUDINARY_URL: process.env.CLOUDINARY_URL
    }
  }]
}
```

## NGINX — reverse proxy + TLS + websockets

### 1. Let's Encrypt (certbot)

```
sudo apt -y install certbot python3-certbot-nginx
sudo certbot --nginx -d api.fitlab.com --agree-tos -m you@domain.com --redirect
```

### 2. Site config `/etc/nginx/sites-available/fitlab-api`:

```
server {
  listen 80;
  server_name api.fitlab.com;
  return 301 https://$host$request_uri;
}

server {
  listen 443 ssl http2;
  server_name api.fitlab.com;

  ssl_certificate /etc/letsencrypt/live/api.fitlab.com/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/api.fitlab.com/privkey.pem;
```

```
# uploads (multer)
client_max_body_size 20m;

# static health
location /healthz { return 200 'ok'; add_header Content-Type text/plain; }

# Socket.IO & API
location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # WebSocket upgrades
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_read_timeout 75s;
    proxy_send_timeout 75s;
    proxy_buffering off;
}

access_log /var/log/nginx/fitlab.access.log;
error_log /var/log/nginx/fitlab.error.log warn;
}
```

```
sudo ln -s /etc/nginx/sites-available/fitlab-api /etc/nginx/sites-enabled/fitlab-api
sudo nginx -t && sudo systemctl reload nginx
```

## MongoDB Atlas (prod)

- Create cluster, DB user, **IP allowlist** (add EC2 public IP/Elastic IP).
- Use SRV connection string in your backend env.
- Enable backups + set indexes for performance.

## S3 — host the React build (private with OAC)

1. Create bucket **fitlab-frontend-prod** (Block Public Access **ON** ).
2. **Build** frontend:

```
npm ci && npm run build
```

3. (CI will upload — manual example:)

```
aws s3 sync build/ s3://fitlab-frontend-prod/ --delete
```

## CloudFront — CDN for frontend (SPA-friendly)

- **Origin** : S3 bucket (use **Origin Access Control (OAC)** ).
- **Behaviors** :
  - Default: GET/HEAD, caching enabled, Gzip/Brotli on.
  - **SPA routing** : Custom error response → for 403/404, respond with `/index.html` and HTTP 200.
  - **TLS** : Attach ACM cert (us-east-1) for `fitlab.com`, `www.fitlab.com`.
  - **Caching** : Long TTL for `/assets/*`, shorter for root.
  - **Logging** : to an S3 logs bucket (optional).

## Route 53 — DNS

- **Hosted zone** for `fitlab.com`.
- **A/AAAA (ALIAS)** :
  - `fitlab.com` → CloudFront dist.
  - `www.fitlab.com` → CloudFront dist.
- **API record** :
  - `api.fitlab.com` → **A** to your **Elastic IP** (recommended), or an **ALB** if you add more EC2s later.

## GitLab CI/CD — pipelines for FE + BE

**.gitlab-ci.yml (starter, no Docker daemon needed):**

```
stages: [test, build_frontend, deploy_frontend, build_backend, deploy_backend, invalidate_cdn]

default:
  image: node:18
  cache:
    key: ${CI_COMMIT_REF_SLUG}
    paths: [node_modules/]

variables:
  AWS_DEFAULT_REGION: "ap-south-1"  # change to yours

before_script:
  - node -v && npm -v

# 1) Tests (optional)
test:
  stage: test
  script:
    - npm ci
    - npm run test --if-present

# 2) Build FE
build_frontend:
  stage: build_frontend
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  script:
    - cd frontend
    - npm ci
    - npm run build
  artifacts:
```

```

    paths: [frontend/build]
    expire_in: 1 week

# 3) Deploy FE to S3
deploy_frontend:
  stage: deploy_frontend
  image: amazon/aws-cli:2.15.0
  needs: [build_frontend]
  script:
    - aws s3 sync frontend/build/ s3://$S3_BUCKET/ --delete
  only: [main]

# 4) Build BE
build_backend:
  stage: build_backend
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  script:
    - cd backend
    - npm ci
    - npm run build
  artifacts:
    paths: [backend/dist, backend/package.json, backend/package-lock.json,
    backend/ecosystem.config.js]
    expire_in: 1 week

# 5) Deploy BE to EC2 (SSH)
deploy_backend:
  stage: deploy_backend
  image: alpine:3.19
  needs: [build_backend]
  before_script:
    - apk add --no-cache openssh-client rsync
    - eval $(ssh-agent -s)
    - echo "$EC2_SSH_PRIVATE_KEY" | tr -d '\r' | ssh-add -
    - mkdir -p ~/.ssh && chmod 700 ~/.ssh
    - ssh-keyscan -H $EC2_HOST >> ~/.ssh/known_hosts
  script:
    - rsync -avz --delete backend/ $EC2_USER@$EC2_HOST:/var/www/fitlab-api/
    - ssh $EC2_USER@$EC2_HOST "cd /var/www/fitlab-api && npm ci --omit=dev && pm2 reload
ecosystem.config.js --update-env"
  only: [main]

# 6) Invalidate CDN
invalidate_cdn:
  stage: invalidate_cdn
  image: amazon/aws-cli:2.15.0
  needs: [deploy_frontend]
  script:
    - aws cloudfront create-invalidation --distribution-id $CLOUDFRONT_DISTRIBUTION_ID --paths
"/*"
  only: [main]

```

Store all secrets in **GitLab CI variables** (masked/protected). Never commit **.env** to Git.

## 🔌 Socket.IO & NGINX (important)

Add these in your NGINX **location** / where the API serves Socket.IO:



```
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
proxy_read_timeout 75s;
proxy_buffering off;
```

In your client, connect to `wss://api.fitlab.com` (if using secure websockets).

## WebRTC nuances (for interviews)

- WebRTC needs **ICE servers** :
  - **STUN** (discover public IP): you can start with Google STUN:  
`stun:stun.l.google.com:19302`.
  - **TURN** (relay when P2P fails behind NAT): run **coturn** on another small EC2 (open 3478 TCP/UDP, and TLS on 5349). Put its credentials in the client ICE list.
- Your backend only handles **signaling** (Socket.IO). Media flows peer↔peer (or via TURN).
- If you skip TURN, some corporate/mobile networks will fail calls — mention this trade-off.

## Security checklist (prod)

- Strong JWT secret; rotate regularly.
- CORS: allow only your FE domains.
- NGINX security headers (`X-Frame-Options`, `X-Content-Type-Options`, `Content-Security-Policy`).
- S3 bucket **private** with **OAC** only.
- Least-privilege IAM for CI.
- UFW + Security Groups minimal.
- Enable **PM2 log rotation** : `pm2 install pm2-logrotate`.
- Optionally add **AWS WAF** in front of CloudFront.

## Observability

- PM2 logs: `pm2 logs`, `~/.pm2/logs`.
- NGINX logs: `/var/log/nginx/*.log` (ship to CloudWatch if you like).
- Add `/healthz` endpoint; Route 53 health checks if you move API behind an **ALB** later.

## Rollouts & rollbacks

- **Frontend** : S3 has **versioning** → quick rollback by restoring the previous build; invalidate CloudFront.
- **Backend** : keep previous build (e.g., `dist_2025-08-19/`). If deploy breaks: `pm2 start <old>` or `pm2 revert`.

## Common pitfalls (quick fixes)

- **CloudFront shows 403/404 on SPA routes** → add custom error response to serve `/index.html` with 200.
- **Socket.IO disconnects** → missing `Upgrade/Connection` headers or proxy timeouts.
- **Uploads fail** → increase `client_max_body_size` in NGINX.
- **API SSL fails** → renew certbot (`sudo certbot renew --dry-run`) and ensure port 80 open for HTTP-01.
- **EC2 IP changed** → attach an **Elastic IP** (or put API behind an **ALB** and point Route 53 to the ALB).

## ----Must Register Domain and Create Hosted Zone for Backend and Start with it

✅ Recommended order (battle-tested)

### 🚩 1) Register the domain + create a hosted zone (now)

- Register `fitlab.com` (Route 53 or GoDaddy).
- If you register outside AWS, update the registrar's **NS records** to Route 53's nameservers.
- Why now?
  - You need the domain active **to issue certificates** (both Let's Encrypt for Nginx and ACM for CloudFront).
  - You can also test DNS + staging subdomains early.

### 💻 2) Backend first: EC2 + Nginx + DNS + SSL

1. **EC2** (Ubuntu) → install Node, PM2, Nginx.
  2. **Elastic IP** : allocate one and associate to your instance (so DNS doesn't break on reboot).
  3. **DNS (Route 53)** :
    - Create **A** (or **ALIAS A**) for `api.fitlab.com` → **your Elastic IP** (or to an **ALB** if you use one).
1. **Let's Encrypt** on EC2:
    - Open SG ports **80/443** .
    - Run `certbot --nginx -d api.fitlab.com` (HTTP-01 challenge hits your instance via the domain).
  2. **Force HTTPS** and reverse proxy to `http://127.0.0.1:3000`.

🔑 For the **API** you typically point Route 53 → **EIP (or ALB)** . You do **not** put CloudFront in front unless you have a reason (API caching/edge auth).

### 📁 3) Frontend: S3 + CloudFront + DNS + SSL

1. **S3** bucket for build artifacts (enable **Block Public Access** ; you'll use **OAC**).
2. **CloudFront** distribution:
  - **Origin** : S3 bucket (with **OAC** ).
  - **Default Root Object** : `index.html`.
  - **Certificate (ACM)** : request in **us-east-1** for `fitlab.com` and `www.fitlab.com`.
3. **Route 53** :

- **A (ALIAS)** `fitlab.com` → your CloudFront distribution.
- **A (ALIAS)** `www.fitlab.com` → same CloudFront distribution.

🔑 CloudFront needs **ACM cert in us-east-1** and uses **CNAME/ALIAS** , not IPs.

#### 🔄 4) Optional: Put CloudFront in front of API

- Not required initially. If you do:
  - **Origin** : `api.fitlab.com` (EC2/ALB).
  - **Behaviors** : e.g., `/api/*` → API origin; `/*` → S3 origin.
  - Tune caching (usually **no cache** or short TTL for APIs) and headers.

#### 🕒 Summary of DNS targets (what goes where)

- **API** (`api.fitlab.com`) → Route 53 **A/ALIAS** to **Elastic IP** (or **ALB** ).
- **Frontend** (`fitlab.com`, `www.fitlab.com`) → Route 53 **A/ALIAS** to **CloudFront distribution** **DNS name** (e.g., `d123abcd.cloudfront.net`).
- **No IPs for CloudFront** — always DNS/ALIAS.

#### 🔒 Certificates: two places, two tools

- **Backend (Nginx on EC2)** → **Let's Encrypt** certs installed on the instance.
- **Frontend (CloudFront)** → **ACM (us-east-1)** cert attached to the distribution.

They're separate and issued/managed differently.

#### ⚠️ Common pitfalls (avoid these)

- 🔄 Waiting to register the domain until the end → then **cert issuance** (Let's Encrypt/ACM) and DNS testing are blocked.
- 🕒 Trying to map Route 53 to a **CloudFront IP** → there isn't one you should use; always ALIAS to the **distribution DNS** .
- 🌐 Requesting the CloudFront cert in the **wrong region** → **must** be **us-east-1** .
- ☐ Not using **Elastic IP** for a single EC2 → A record breaks when public IP changes.
- 🔒 Leaving S3 public when using CloudFront → use **OAC** + bucket policy to allow only CloudFront.

#### 🧐 A clean, minimal sequence you can follow

1. **Buy domain** (`fitlab.com`) → create **hosted zone** → fix NS at registrar (if external).

2. **Backend** :

- EC2 (t3.micro) + **Elastic IP**
- Nginx reverse proxy to Node
- Route 53: `api.fitlab.com` → **A to EIP**
- **Let's Encrypt** on EC2

1. **Frontend** :

- Build React → upload to **S3**
- **CloudFront** with **OAC** + **ACM (us-east-1)** for `fitlab.com`, `www.fitlab.com`
- Route 53: `fitlab.com`, `www.fitlab.com` → **ALIAS** to CloudFront

1. (Optional) Add `/api/*` behavior in CloudFront to your API origin later.

.....

.....

## ----Backend URL format convention

### 1. Usual URL format for backend

For modern websites, the most common convention is to **separate frontend and backend with subdomains** :

- **Frontend (public site):**

`https://fitlab.com` or `https://www.fitlab.com`

- **Backend API:**

`https://api.fitlab.com`  (most widely used in industry)

or

`https://backend.fitlab.com` (also fine, just less common)

👉 `www.fitlab.backend.com`  is **not a good choice** . That's a *third-level subdomain* under `backend.com`, which makes things messy and non-standard. Stick with `api.fitlab.com` or `backend.fitlab.com`.

### 2. Mapping your existing local routes

Right now your backend runs on:

```
http://localhost:3000/support/video
```

When deployed, if you choose `api.fitlab.com` (or `backend.fitlab.com`), this becomes:

```
https://api.fitlab.com/support/video
```

So yes — just replace `localhost:3000` with your chosen backend domain/subdomain.

### 3. Frontend calling backend

- Your React frontend (hosted on S3 + CloudFront) will be served at `fitlab.com` (or `www.fitlab.com`).
- In your frontend code, API calls should be directed to `https://api.fitlab.com/....`

This way you **separate concerns** :

- Users interact with `fitlab.com`.
- The app fetches data from `api.fitlab.com`.

### 4. SSL & CORS

- You'll need SSL certificates for both `fitlab.com` and `api.fitlab.com` (use **AWS Certificate Manager** if you're deploying via Route 53 + CloudFront).
- Ensure CORS is configured on your backend so your frontend (`fitlab.com`) can make requests to `api.fitlab.com`.

#### Best practice recommendation for FitLab:

- Frontend → `fitlab.com`
  - Backend → `api.fitlab.com`
- .....
- .....

## ----S3 Hosting in Detail

### 1. Build folder for S3

You **do not** upload your whole `Frontend/` folder to S3.

Instead, you need to create a **production build** of your frontend (since you are using Vite + React).

Run this inside your `Frontend/` directory:

```
npm run build
```

This will generate a **dist/** folder (by default in Vite).

That **dist/** folder is what you upload to your **S3 bucket** (not the entire **Frontend/** source).

So workflow is:

- You develop inside **Frontend/**.
- Run **npm run build**.
- Upload only the contents of **dist/** to S3.

## 2. .env file in Frontend

The **.env** file in frontend is **not directly used in production** .

- When you run **npm run build**, Vite **reads the .env file at build time** and replaces variables in your code with their values.
- After building, those variables get “baked” into the final JS files inside the **dist/** folder.

👉 That means:

- You **don't upload .env to S3** .
- Instead, ensure that your **.env** file is properly configured (e.g., API URLs like **VITE\_API\_URL=https://api.fitlab.com**).
- Then run the build so that those values are compiled into the app.

## 3. S3 Hosting Checklist

When uploading to S3:

1. Create a bucket named exactly like your domain (e.g. **www.fitlab.com**).
2. Enable **static website hosting** in bucket properties.
3. Upload **dist/** contents.
4. Set permissions → either make it public or (recommended) serve it via **CloudFront** for SSL + caching.
5. Connect your domain (**www.fitlab.com**) in Route 53 to CloudFront, not directly to S3.

✅ So in short n:

- You don't upload **Frontend/**, you upload the **dist/ build** .
  - You don't upload **.env**, you just use it during the build step.
- 
-